

Abstract

Unmanned aerial vehicles have become essential tools in search and rescue operations due to their capability to safely access hazardous zones while providing real-time situational awareness. Deploying drones in complex urban environments requires advanced path planning systems that can optimize flight routes while responding to dynamic environmental conditions.

The A* algorithm and reinforcement learning (RL) based approaches have been proposed for drone path planning and achieved promising results. However, the A* algorithm fails to adapt when environmental conditions transform during mission operations. The training process of pure RL-based methods needs long periods of time to produce reliable results while using many samples. Moreover, the unpredictable movements of dynamic obstacles require real-time response, making it even more difficult for existing approach.

To address the above problems, in this dissertation, we propose a hybrid framework for efficient and real-time path planning. The hybrid framework combines A* and reinforcement learning, where A* generates the initial path plan and RL optimize the plan to adapt to dynamic environment. To optimize the efficiency, we further propose *Experience Replay Buffer* which saves previous experiences for future learning through random batch selection to eliminate time-based dependencies and produce multiple learning opportunities from each experience.

Even with the hybrid framework, the successful rate of path planning is still limited. To further enhance path planning efficiency and effectiveness, we introduce a *Multi-Level RL Policies* through a three-agent hierarchical structure which uses Main RL for strategic heuristic guidance, Local RL for instant obstacle clearance and Global RL for essential route optimization. RL-Based Obstacle Prediction is to forecast upcoming obstacle locations based on movement patterns in pattern-based dynamic environments.

We evaluated the proposed approach in a 2D grid simulation environment, and compare with SOTA approach D*. Evaluation results show the algorithm achieves 100% success in static environments through its ability to generate optimal paths. The system achieves 93.3% success rate when operating in environments with moving obstacles whereas A* and D* Lite achieve 30.0% and 73.3% success rates respectively. The obstacle prediction method in pattern-based

dynamic environments stops 87.5% of dynamic obstacle collisions. The system reaches its highest energy conservation rate of 15.1% when battery reserves drop to their lowest point. The system achieves a 41.2% reduction in training time through its application of transfer learning methods. The system achieves successful global optimization through real-time local adaptation which creates an operational system for autonomous search and rescue operations that provides quick response times and adaptable mission performance.

Keywords: Drone path planning; A* algorithm; Reinforcement Learning; MR-QLearning; dynamic obstacle avoidance; unmanned aerial vehicle; hybrid path planning; search and rescue

Contents

Chapter 1 Topic Introduction	1
1.1 Source	1
1.2 Background	1
1.3 Significancy	3
1.4 Key Research Problems	4
1.5 Thesis Structure	7
Chapter 2 Related Work	9
2.1 Machine Learning Based Algorithm Systems	9
2.2 A-Star Algorithm for Drone System	18
2.3 Search and Detection Operation in Drone System	22
Chapter 3 Path Planning with Hybrid A* and Reinforcement Learning	26
3.1 System Overview	26
3.2 Static Global Path Planner Based on A* Algorithm	27
3.2.1 Heuristic Function Design	28
3.2.2 Battery-Aware Heuristic Enhancements	28
3.3 Adaptive Navigator Via MR-Qlearning (Modified Reward)	29
3.3.1 Base Q-Learning Framework	30
3.3.2 Experience Replay Buffer	33
3.3.3 Uncertainty Quantification	35
3.3.4 Adaptive Confidence Threshold	37
3.3.5 Transfer Learning Framework	38
3.3.6 Mathematical Formulations	39
3.3.7 Training Parameters	41
3.3.8 Convergence Analysis of MR-QLearning	42
3.4 Path Optimization by Combining A* and MR-QLearning Approach	43
3.4.1 RL-Guided Heuristic Learning	43
3.4.2 RL-Based Path Refinement	45

3.4.3 Computational Complexity Analysis	45
3.4.4 Proposed Hybrid Approach Flowchart	46
Chapter 4 Multi-Level RL Integration for Path Optimization	49
4.1 System Architecture	49
4.2 Three-Agent Hierarchy	49
4.2.1 Main RL Agent	49
4.2.2 Global RL Agent	50
4.2.3 Local RL Agent	50
4.2.4 Agent Coordination and Cascade Mechanism	51
4.3 Mathematical Framework of the Multi-Level RL	51
4.3.1 Shared State and Action Space	52
4.3.2 Agent-Specific Q-Value Update	52
4.3.3 RL-Guided Heuristic Integration	52
4.4 RL-Based Path Refinement	53
4.5 A* Fallback Mechanism	53
4.6 RL-Based Obstacle Prediction	54
4.6.1 Prediction Operational Mechanism	55
4.6.2 Adaptive Navigation	55
4.7 Continuous Learning During Execution	56
4.7.1 Detailed Agent Interaction Protocol	57
4.7.2 RL-Based Obstacle Prediction Mechanism	58
4.7.3 Training Budget Allocation	59
Chapter 5 Experiment and Results	60
5.1 Experimental Setup	60
5.1.1 Environmental Configuration	60
5.1.2 Methods Evaluated	61
5.1.3 Ablation Study	63
5.1.4 Statistical Analysis	63
5.1.5 Dynamic Environment Evaluation	63
5.1.6 Metrics	64

5.2 Experimental Results	65
5.2.1 Performance Comparison in a Static Environment	66
5.2.2 Performance Comparison in a Dynamic Environment	67
5.2.3 Ablation Study Results	68
5.2.4 Battery-Aware Routing Results	69
5.2.5 Comprehensive Experiments	70
5.2.6 Obstacle Prediction Method Results	72
5.2.7 Multi-Level RL Results	73
5.2.8 Statistical Significance Analysis	75
5.2.9 Sensitivity Analysis	76
5.3 Discussion	77
5.3.1 Interpretation of Key Findings	77
5.3.2 Comparison with Existing Literature	78
5.3.3 Practical Deployment Considerations	78
Chapter 6 Conclusion and Future Work	80
6.1 Summary of Contributions	80
6.2 Limitations and Constraints	80
6.2.1 Future Research Directions	81
6.2.2 Conclusion	82
References	84
Acknowledgements	89

List of Figures

Figure 2.1 Concept of Drone Location and Id Information Sharing System	12
Figure 2.2 Environment information [57]	14
Figure 2.3 An illustration of 32-Neighbor A* search algorithm. [59]	16
Figure 2.4 System model [63]	19
Figure 2.5 Path selection during search [64]	20
Figure 2.6 Radar detection probability of different algorithms in the third threat scenario [65]	21
Figure 2.7 A search and detection autonomous drone system: from design to implementation [67]	22
Figure 2.8 Field of view (FOV) projection and footprint extent of a vision-based sensor.[69]	24
Figure 3.1 Comprehensive flowchart of the Hybrid A* + Reinforcement Learning system illustrating the four operational phases	27
Figure 3.2 Mr-QLearning Algorithm Flowchart	30
Figure 3.3 Experience Replay Buffer Mechanism	34
Figure 3.4 Uncertainty Quantification Mechanism Chart	36
Figure 4.1 RL-Based Obstacle Prediction flowchart	54
Figure 5.1 Example of The Grid Environment	64

List of Tables

Table 3.1	MR-QLearning Training Parameters	41
Table 3.2	Convergence Metrics Under Different Configurations	43
Table 3.3	Computational Complexity Summary	46
Table 4.1	Multi-Level RL Agent Configuration Parameters	51
Table 4.2	Cascade Decision Logic and Trigger Conditions	54
Table 4.3	Obstacle Density Classification	56
Table 4.4	Obstacle Prediction Accuracy by Movement Type	59
Table 4.5	Agent Training Budget Allocation	59
Table 5.1	Key Metrics for Path Planning Performance Evaluation	66
Table 5.2	Performance Comparison (50 trials, 20×20 grid, 15% obstacle density)	66
Table 5.3	Dynamic Environment Performance Comparison	67
Table 5.4	Ablation Study Results	68
Table 5.5	Battery-Aware Routing Performance	69
Table 5.6	Performance Across Static Only Obstacle Densities	70
Table 5.7	Performance Across Grid Sizes (15 trials per condition)	71
Table 5.8	Transfer Learning Performance (15 trials)	71
Table 5.9	Obstacle Prediction Method Performance	72
Table 5.10	Multi-Level RL Architecture Ablation Study Results	73
Table 5.11	Statistical Significance of Dynamic Environment Comparisons	76
Table 5.12	95% Confidence Intervals for Hybrid Approach Performance	76
Table 5.13	Hyperparameter Sensitivity Analysis Results	77

Chapter 1 Topic Introduction

1.1 Source

This research develops a Hybrid A* + Reinforcement Learning (RL) system which performs intelligent real-time path planning and optimization for autonomous drone search and rescue operations. The main goal involves creating an efficient system which allows drones to operate independently through urban areas containing fixed and moving barriers while achieving maximum battery performance. The system unites A* algorithm optimality with reinforcement learning adaptability through its new hybrid system design. The A* algorithm delivers precise path discovery through its ability to generate the most efficient routes while MR-QLearning serves as an improved Q-learning system which enables the drone to acquire knowledge from its experiences and handle dynamic situations in real-time. The simulation uses Python to create a 2D grid which represents the environment while it models obstacles and terrain characteristics and environmental dynamics. The system achieves three main goals through its operation which includes real-time path planning with dynamic obstacle avoidance and battery-aware routing for energy efficiency and adaptive navigation for different terrain conditions and obstacle densities.

1.2 Background

Drones have become transformative tools in various civilian and emergency response applications. Their capabilities enormously enhance the efficiency of operations, demonstrating their potential to contribute positively to society. These autonomous or remotely piloted aircraft systems offer mobility, flexibility [1], and aerial perspective capabilities, making them particularly valuable in time-critical scenarios such as search and rescue (SAR) operations. The integration of drones in SAR operations has shown remarkable potential [1–3] to enhance the efficiency and effectiveness of emergency response efforts, especially in scenarios where traditional ground-based search methods face significant limitations or risks that the drones cannot be affected by.

Search and rescue missions are challenging and time-sensitive missions aimed at finding and rescuing people in distress or imminent danger. The conventional approach to SAR operations, which primarily relies on ground teams and human-crewed aircraft, often faces challenges such as limited visibility, difficult terrain accessibility, and significant resource requirements regarding personnel and equipment.

Through their unique characteristics, drones present a compelling solution to many challenges. Their ability to rapidly deploy, operate in challenging environments, provide real-time aerial surveillance, and operate in a timely manner makes them invaluable assets in SAR missions. Moreover, the reasonably low operational costs and reduced risk to human personnel make drones attractive for emergency response organizations. However, effectively utilizing drones in SAR operations requires educated path planning and optimization capabilities [4] to maximize their potential impact.

Path planning, in the context of drone operations, refers to locating and determining the optimal route for a drone to navigate through from its current position to a target location while considering various constraints and objectives. These constraints typically include obstacle avoidance, battery limitations, and environmental conditions which affect the mission's success. In SAR scenarios, effective path planning becomes particularly crucial as it directly impacts the probability of successfully locating the operations targets in the shortest possible time while managing limited resources efficiently [5].

The complexity of drone path planning and optimization in SAR missions is heightened by several factors[6]:

1. Real time Decision making: The need to process and respond to dynamic environmental information [7, 8] and changing mission parameters in real-time
2. Resource optimization and balancing: managing limited battery life while maximizing search coverage
3. Environmental challenges: navigating through complex urban or natural environments with various obstacles.
4. Target uncertainty: operating with incomplete or uncertain information about the target precise location.

5. Time Criticality: balancing thorough search patterns with the urgency of the mission

Traditional path-planning approaches often struggle to address these challenges systematically, particularly in complex urban environments with numerous obstacles and highly constrained search space. While existing research has made significant progress in individual aspects of drone navigation, there remains a crucial need for integrated solutions that can handle real-time path planning while optimizing it.

Recent achievements in artificial intelligence and autonomous systems have opened new possibilities [9, 10] for enhancing drone capabilities in SAR operations. Particularly, developments in areas such as reinforcement learning and computer vision have shown promising results [11, 12] in improving autonomy and effectiveness of drone operations. However, the integration of these technologies and methods into a cohesive system capable of real-time operation in complex environments remains a significant challenge enhance [9, 13].

This research addresses these challenges by developing an intelligent real-time path planning [14] and optimization system designed explicitly for drone-based SAR missions.

The proposed system aims to improve SAR operations Through:

- Real-time path planning[15] with dynamic obstacle avoidance capabilities.
- Battery-aware routing algorithms that maximize search coverage in the targeted area.
- Adaptive navigation strategies for complex urban environments.

1.3 Significancy

The military and civilian sectors now depend on drones because these systems provide flexible operation and multiple operational benefits. The military uses drones to protect personnel from dangerous situations during reconnaissance and surveillance operations [16, 17] while achieving better cost performance than manned aircraft systems [15, 18]. The unmanned systems operate in dangerous zones while maintaining long surveillance periods and accessing areas which human operators cannot reach [18] to deliver tactical benefits through immediate intelligence collection and safe operations in hostile areas [18].

The use of drones has transformed various industries through their ability to perform fast damage assessments and supply emergency aid during natural disasters [17, 18]. The

combination of safety and operational efficiency enables drone systems to inspect bridges and power lines and tall structures while they revolutionize agricultural practices through precise crop tracking. Emergency services now depend on drone technology to perform firefighting duties and law enforcement work and search and rescue operations while environmental protection benefits from improved wildlife tracking and environmental assessment tools [17]. Drone technology serves multiple domains through its applications in urban planning which include aerial mapping and urban development monitoring [17, 19].

The essential role of drones in search and rescue operations becomes evident through multiple essential elements. The system provides fast coverage of extensive areas which ground teams need extended periods to search [16, 17] and enables access to dangerous and distant locations that ground teams cannot reach [18] and provides aerial views that help detect hidden patterns and clues [19]. The fast deployment of drones enables quick response times because they can arrive at search sites within short periods [15, 19] while providing instant visual data transmission to rescue teams [17, 18] and maintaining continuous search operations under suitable weather conditions [15, 18]. The system provides resource optimization through its affordable operations which outperform manned aircraft [15] and enables better ground team deployment through drone observations [15, 18] and protects personnel from dangerous situations [17]. The search process becomes more effective because drones can use multiple sensors to detect different search environments [19] while following predetermined search patterns to maximize coverage [15] and automatically recording searched areas and discovered evidence for operational coordination and documentation purposes [17].

1.4 Key Research Problems

The research solves several critical issues in autonomous drone path planning for search and rescue missions through the implementation of a Path optimization A* and reinforcement learning system.

1. **Dynamic Obstacle Navigation in Real-Time** The research addresses the critical challenge of navigating through shifting obstacles through its integration of A* pathfinding with an improved reinforcement learning system. The A* algorithm which

exists in its traditional form operates only in environments without movement because it fails to detect the moving obstacles which search and rescue missions encounter. The Hybrid system [20] implements three separate solutions to handle this issue: A* real-time replanning becomes active when obstacles appear in the planned route which results in a fast response time and RL learning operates throughout execution to update Q-values based on drone movements for better adaptation to changing situations and RL-based obstacle prediction which monitors movement patterns to predict upcoming obstacle locations for early avoidance. The system uses multiple instance RL agents which operate with two distinct policies for global and local control—the Local RL agent performs immediate obstacle avoidance with quick responses, and the Global RL agent handles significant path modifications when local adjustments prove inadequate. The system uses this hierarchical method to reach high success rate in changing environments which results in a better performance than basic A*[21].

2. **The Need for Optimality and Adaptability** The research develops a hybrid system which solves the main problem of discovering the best routes while preserving environmental adaptability. The A* algorithm produces the most efficient routes [22, 23] but it becomes unable to find new paths when moving obstacles start blocking its originally planned route. The flexibility of pure reinforcement learning methods exists but these methods fail to reach their best performance while needing long periods of training. The Hybrid system solves this conflict by using RL to guide heuristic learning which enables Q-values to modify A* search heuristics through a continuous exchange of knowledge between classical and learning-based planning methods. The system uses RL-based path refinement to enhance A* paths through learned preference application which maintains A*'s optimality benefits while RL adapts to new situations. The system unites A* optimization with RL adaptability to create an entire solution which handles unpredictable search and rescue scenarios.
3. **Inefficiency in Pure Reinforcement Learning approach** The research addresses the main issue of sample inefficiency in pure reinforcement learning through four improvement additions to the MR-QLearning algorithm. The Experience Replay

Buffer[24] stores past experiences which it uses to select random batches for additional learning to break temporal correlations and enable the agent to learn from each experience multiple times instead of just once. The system uses Uncertainty Quantification to monitor Q-value changes during updates because it identifies unstable system states while generating confidence metrics from visit statistics and Q-value statistical properties and absolute values. The Adaptive Confidence Threshold operates as a third component which modifies the RL to A* transition point through performance-based adjustments. The system decreases the threshold value to use A* more frequently when RL experiences performance difficulties but increases the threshold value to depend on RL decisions when RL achieves good results. The Transfer Learning Framework allows knowledge sharing between environments [25, 26] with similar characteristics through Q-table export from source environments and compatibility verification and new environment setup with acquired knowledge. The Real-Time Optimization mode reduces training episodes by 85% to 80 episodes while preserving high success rates which enables the system for immediate field use.

4. **Battery Efficiency and Energy-Aware Path Planning** The research addresses battery efficiency problems through Battery-Aware Routing which merges energy awareness into A* and RL algorithms. The system uses a progressive urgency model which bases its priority decisions on battery level as a dynamic factor that affects mission planning throughout the operation. The A* component [27, 28] uses two adjustment mechanisms which depend on remaining battery power: the system uses default costs when battery levels remain high but it increases heuristics and step costs when battery levels reach warning thresholds to select shorter paths. The MR-QLearning component uses a reward function which includes battery penalties that increase based on how much energy the system uses during each step. The reward function includes two types of penalties which depend on battery usage per step and two additional penalties that activate when battery levels reach specific critical points. The RL agent learns to manage its goal pursuit activities while saving power through this system which produces automatic drone

behavior that uses short routes when the battery runs out but explores freely when the battery level is sufficient.

5. **Robustness to Environmental Uncertainty** The research implements Adaptive Navigation together with a defense-in-depth architecture to enhance system resistance against environmental uncertainties [29] through multiple reliability layers. The Adaptive Navigation component performs environment analysis at two different levels which include terrain difficulty assessment to determine local obstacle numbers for each cell and navigation cost increases when cells become congested. The system uses two methods to analyze environment complexity: terrain difficulty analysis determines local obstacle numbers for each cell which affects navigation costs through congested areas and obstacle density awareness determines total environment complexity to select appropriate planning methods [30]. The system uses an advanced fallback system which maintains mission success: The system starts with Local RL for quick adaptation when it faces obstacles; Global RL takes over for complete replanning when local methods prove ineffective; The system uses A* pathfinding as its last resort when all previous planning methods fail. The system uses multiple instance RL architecture with separate Q-tables for Main and Global and Local agents to achieve redundancy and the Uncertainty Quantification mechanism stops the system from executing untrustworthy learned policies. The system design with multiple layers of protection enables successful mission execution in different operational settings which range from controlled static environments to unpredictable dynamic search and rescue operations[20].

1.5 Thesis Structure

This thesis is organized into six chapters as follows:

Chapter 1: Topic Introduction presents the research background, significance, and key research problems that motivate the development of the Hybrid A* + Reinforcement Learning path planning system for autonomous drone search and rescue operations.

Chapter 2: Related Work reviews existing literature on machine learning based algorithm systems, A* algorithm applications for drone systems, and search and detection

operations in drone systems, identifying research gaps that establish the foundation for the proposed approach.

Chapter 3: Path Planning with Hybrid A* and Reinforcement Learning details the core methodology including the A* path planner with battery-aware heuristics, the MR-QLearning algorithm with its four novel enhancements, the hybrid path optimization approach, and the RL-based obstacle prediction mechanism.

Chapter 4: Multi-Level RL Policies for Path Optimization presents the three-agent hierarchical architecture with Main, Global, and Local RL agents, the cascade coordination mechanism, and the mathematical framework for RL-guided heuristic integration.

Chapter 5: Experiment and Results provides comprehensive experimental evaluation including performance comparisons in static and dynamic environments, ablation studies, battery-aware routing results, and statistical significance analysis.

Chapter 6: Conclusion and Future Work summarizes the research contributions, discusses limitations and constraints, and identifies future research directions for extending the proposed system.

Chapter 2 Related Work

2.1 Machine Learning Based Algorithm Systems

Q-Learning [31–33] represents a model-free RL algorithm which teaches action-value functions $Q(s, a)$ to predict the total reward that results from performing action a in state s and then using the optimal policy for subsequent decisions[34]. The algorithm performs Q-value updates through temporal difference learning which depends on observed rewards together with future Q-values. The Q-value update follows the Bellman equation[35]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.1)$$

The update rule contains three main components which are α for learning rate between 0 and 1 and γ for discount factor between 0 and 1 and r for immediate reward and s' for next state[36]. The off-policy algorithm Q-Learning operates through an exploration policy (ϵ -greedy) which enables it to discover the best policy[37, 38].

Double Q-Learning [39, 40] solves the overestimation problem which Q-learning experiences because the max operator in its update process leads to excessive action value overestimation. The system operates with two separate Q-tables named Q_1 and Q_2 which serve for action selection and evaluation assessment. The update process moves between the two tables:

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha[r + \gamma Q_2(s', \arg \max_{a'} Q_1(s', a')) - Q_1(s, a)] \quad (2.2)$$

The separation of action selection from evaluation processes helps to minimize overestimation errors which results in more stable learning outcomes. The research shows Double Q-Learning outperforms Q-Learning in terms of speed and policy reliability when operating in stochastic environments.

Experience Replay [41] became well known through its use in Deep Q-Networks (DQN) which Mnih et al. presented in their work. The authors used the Deep Q-Network (DQN) architecture from [42] for deep reinforcement learning. The method uses a replay buffer to store previous experiences which include state information and action selection

and reward values and next_state data before it selects random training batches. The system breaks down time-based relationships between training data points which enables the agent to acquire knowledge from experiences that took place at various points in time. The system achieves better sample efficiency through experience replay because it allows the system to use experiences more than once and it enhances learning stability by making gradient estimates less volatile.

Experience replay serves as a critical element for deep RL because neural networks need diverse uncorrelated training data to learn stability. The technique of experience replay serves mainly deep RL systems which use function approximation because it stores data in a buffer to eliminate dependencies between consecutive input sequences.

Neural A* Neural A* [43] employs neural networks to discover enhanced heuristics which improve A* search performance. The method trains a neural network on successful paths to predict more accurate heuristic values than standard distance-based heuristics. The learned heuristics enable A* search to operate more efficiently because they help the algorithm explore fewer nodes while still ensuring it finds the optimal solution.

The neural network uses current state and goal information to generate an enhanced heuristic value as its output. The training process occurs independently from a collection of successful routes which produces heuristics that guide the online pathfinding process. Neural A* outperforms standard A* with Manhattan distance heuristics because it achieves better search efficiency.

Battery-Aware Routing Research on battery-aware routing has received substantial attention in wireless sensor networks and mobile robotics because these systems require immediate solutions for their critical energy limitations [44, 45]. The majority of research methods implement fixed cost functions that base their route choices on battery power status. The system uses specific methods which raise path expenses when battery levels become low to direct traffic through shorter routes (Singh et al., The research of Liu and Sun (2014) focuses on battery usage patterns for various terrain conditions.

The main objective of battery-aware routing in wireless sensor networks involves distributing network traffic to achieve maximum network operation duration [46, 47]. Mobile

robotics systems implement battery level as a constraint to optimize their paths because they select routes which consume less energy when their battery power reaches minimum levels [48, 49]. The majority of current methods implement static cost functions which fail to adjust their operation according to battery power status [50, 51]. The research fills this knowledge gap by implementing step-by-step battery-aware optimization methods which modify A* heuristics and RL reward functions [52].

Real time 3D Path Planning for UAV Navigation Tullu et al. [8] proposed a machine-learning approach to real-time 3D path planning for autonomous UAV navigation in cluttered environments. The authors approach involved integrating YOLO (You Only Look Once) object detection algorithm with a lightweight 3D path planner to enable real-time obstacle avoidance. Their approach differs from traditional path planning algorithms like A* by avoiding exhaustive node exploration and instead mapping the UAV's current location to select nodes between consecutive obstacles. The system uses a ZED mini stereo camera for object detection and localization, combined with GPS and LiDAR for UAV positioning. The path planner considers both open obstacles (requiring passage through, like windows) and closed obstacles (requiring avoidance maneuvers), applying a heuristic function for cost minimization. The effectiveness of their approach was through Software-In-The-Loop (SITL) simulations tests in Gazebo environments and preliminary real-world flight tests. Results demonstrated successful autonomous navigation through various obstacles, including pedestrians, windows, poles, tunnels, and trees. An essential contribution is decreasing the computational burden on the companion computer by integrating fast object detection with efficient path planning, making it suitable for real-time applications. However, limitations included GPS accuracy issues in open fields and the need for further real-world testing in more complex environments.

Data Analysis and Modeling for Drone Communications using ML algorithms Shan, R. Miura, T. Kagawa [53] presented a research paper on unmanned aerial vehicle (UAV), also called a drone, is getting increasingly important in many emerging technology areas. For communication area, the drone also takes an important role in lots of significant topics like emergency communications, device-to-device (D2D) communications, and the Internet of Things (IoT). One of the important drone applications is to collect and share data among



Figure 2.1 Concept of Drone Location and Id Information Sharing System

drones and other aircraft, which is useful for drone control so that dangerous conditions can be avoided. The conceptual framework for drone location and ID information sharing among different aircraft types is illustrated in Figure 2.1. In particular, the drone control and safety guarantees are difficult to attain, especially, when drones fly beyond the line of sight (BLOS). For this reason, the authors develop a drone location information sharing system using the 920-MHz band. they use this system to do a long distance propagation field experiment for model establishment. Unfortunately, the current data collection for model establishment work needs a great effort and time to do experiments to collect a huge number of data for data analysis so that a suitable model can be established. Therefore, in this paper, the authors propose a novel method, which is based on machine learning approach, to data analysis and model establishment for drone communications, so that the effort and cost for establishing model can be reduced and a model, which captures more details about the drone communications, can be obtained. The results of this paper validate that the proposed method can indeed establish a more complicated model with less effort. Specifically, from the distribution of the training error, it can be known that there are over 80% training errors with intensity less than 5, which ensures the error performance of the proposed method.

Advanced implementation in drone for Smart City Developments using ML Algorithm Sampath Boopathi [54] presented a chapter that explores the integration of drones, machine learning, and artificial intelligence (AI) in smart city development. Drones can revolutionize urban planning, energy efficiency, noise reduction, environmental monitoring,

traffic management, infrastructure inspection, public safety, data security, and privacy protection. AI-driven solutions enable data-driven decision-making for resource allocation, sustainability, and predictive modeling. AI optimizes flight paths for energy efficiency, noise reduction strategies enhance drone social acceptance, and autonomous drone navigation is crucial for safe urban deployment. Drones optimize traffic flow, reduce congestion, and enhance safety. AI provides predictive insights into traffic patterns, and drones aid in law enforcement, emergency response, and first responder support. Data security and privacy protection measures are essential for maintaining public trust.

Sound-Based Amateur Drone Detection for Public Safety Applications Muhammad Zohaib Anwar, [55] presented that In recent years, popularity of unmanned air vehicles enormously increased due to their autonomous moving capability and applications in various domains. This also results in some serious security threats, that needs proper investigation and timely detection of the amateur drones (ADr) to protect the security sensitive institutions. In this paper, the author propose the novel machine learning (ML) framework for detection and classification of ADr sounds out of the various sounds like bird, airplanes, and thunderstorm in the noisy environment. To extract the necessary features from ADr sound, Mel frequency cepstral coefficients (MFCC), and linear predictive cepstral coefficients (LPCC) feature extraction techniques are implemented. After feature extraction, support vector machines (SVM) with various kernels are adopted to accurately classify these sounds. The experimental results verify that SVM cubic kernel with MFCC outperform LPCC method by achieving around 96.7% accuracy for ADr detection. Moreover, the results verified that the proposed ML scheme has more than 17% detection accuracy, compared with correlation-based drone sound detection scheme that ignores ML prediction.

Reinforcement learning framework for UAV SAR For optimization of path Ramezani and Amiri Atashgah [56] displayed an energy-aware hierarchical reinforcement learning (HRL) framework for unmanned air vehicle (UAV) search and rescue missions that integrate LSTM-based battery consumption prediction for optimizing path planning. Their framework involves a high-level controller (HLC) that sets feasible goals based on predicted battery requirements and a low-level controller (LLC) that executes navigation commands. A novel

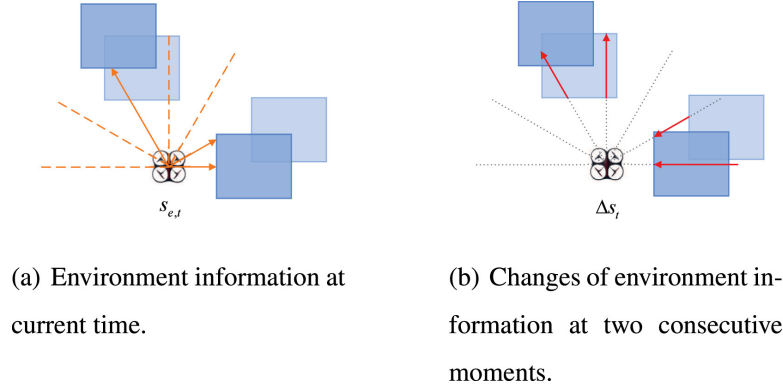


Figure 2.2 Environment information [57]

innovation is using a bidirectional LSTM model to accurately predict battery consumption for different targets, helping the HLC make energy-efficient decisions. The framework includes hindsight experience replay (HER) to improve learning efficiency and adaptability. The system was validated through extensive simulations in MATLAB using four test environments with varying obstacle densities and target counts. Results demonstrated superior performance compared to baseline methods, achieving 90-95% success rates across environments while maintaining low collision rates of 3-8%. The authors' hierarchical approach with LSTM battery prediction enabled more efficient path planning and energy utilization compared to traditional reinforcement learning methods. Key advantages included setting achievable goals based on energy constraints and adapting to dynamic environments while optimizing battery usage through predictive insights. Test results verified the framework's effectiveness for autonomous UAV navigation in complex search and rescue scenarios.

Deep learning approach in a multi-obstacle environment In this study, Zhang et al. [57] displayed a deep reinforcement learning (DRL) method for autonomous UAV navigation in dynamic multi-obstacle environments. The environment information representation, including obstacle positions at consecutive timesteps demonstrating movement patterns, is shown in Figure 2.2. The key methodological contributions include 1) the Development of a two-stream Actor-Critic network structure based on Twin Delayed Deep Deterministic Policy Gradients (TD3) that considers both current observations and changes in environment observations to predict obstacle movements, 2) the Implementation of a Unity3D simulation platform to reduce the reality gap, and 3) Design of a comprehensive reward function incorporating distance, step penalty, angle penalty and obstacle avoidance components. The method was evaluated

through extensive experiments in both static and dynamic environments with varying obstacle configurations. Results demonstrated superior performance compared to baseline methods:

- In static environments with random obstacles, their method achieved up to 95.25% success rate compared to 92.10% for DDPG and 95.15% for conventional TD3
- In dynamic environments with moving obstacles, their approach maintained a 77.65% success rate even at high obstacle speeds (20 m/s), while DDPG dropped to 59.80%
- The method showed better generalization ability and faster convergence during training compared to baselines.
- The two-stream network structure enabled effective prediction of obstacle movements, allowing proactive collision avoidance.
- Ablation studies on reward function design validated their formulation's ability to balance safety and efficiency

Air-to-Air Simulated Drone Dataset for AI-powered problems The Air-to-Air Simulated Drone Dataset (A2A-SDD) [58] significantly contributes to counter-UAV research through its comprehensive simulation-based approach to data generation [58]. Using the AirSim platform, this dataset provides diverse drone interaction scenarios, featuring up to three monitoring drones pursuing one or two target drones in various environmental conditions [58]. The dataset's architecture incorporates five different drone types, including several DJI models (Phantom, Mavic, FPV, and Inspire) alongside a generic quadrotor model, offering a broad representation of common UAV platforms [58]. A key innovation of A2A-SDD lies in its multi-view data collection methodology [58]. The dataset provides richer spatial information by capturing scenarios from multiple camera perspectives than traditional single-view approaches. This multi-perspective design is complemented by comprehensive annotations, including precise XYZ coordinates for all drones, detailed bounding box information, and payload status indicators [58]. The dataset also incorporates various environmental conditions, simulating sunny, rainy, and snowy weather across different terrains, from natural landscapes to urban environments [58]. The practical utility of A2A-SDD has been demonstrated through experimental validation using deep learning models [58]. The researchers achieved notable results in two critical areas: distance estimation and payload detection. The distance estimation

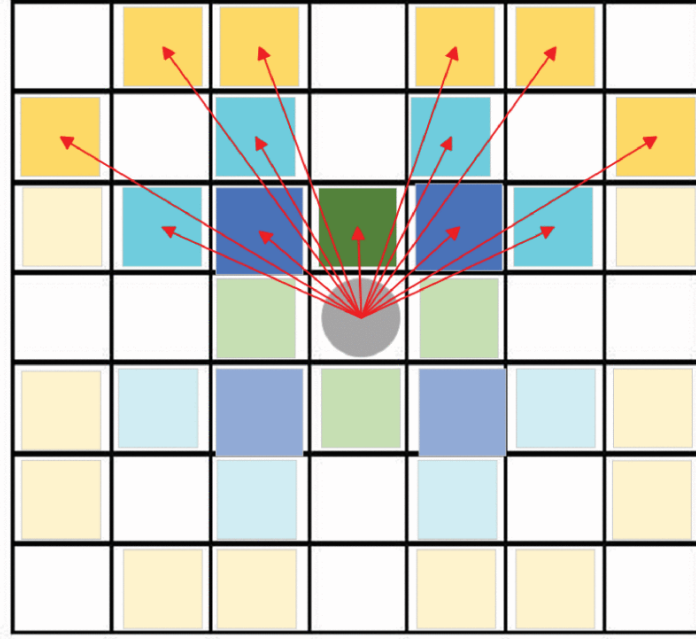


Figure 2.3 An illustration of 32-Neighbor A* search algorithm. [59]

models achieved an RMSE of approximately 8.4 meters when tested on real-world data, while payload detection systems demonstrated accuracies between 87-92% in practical applications [58]. These results validate the dataset's effectiveness in bridging the gap between simulated training data and real-world applications. However, A2A-SDD does face certain limitations [58]. The reliance on simulation, while enabling comprehensive data collection and annotation, raises questions about fully representing real-world complexities. The current focus on visual data also allows expansion into other sensing modalities [58]. Despite these limitations, the dataset is valuable for developing and testing counter-UAV measures, particularly in scenarios where real-world data collection would be impractical or cost-prohibitive. The development of A2A-SDD marks a significant step forward in counter-UAV research, providing researchers with a robust platform for developing and testing detection, tracking, and classification algorithms [58]. The dataset's comprehensive nature, combined with its validated effectiveness in real-world applications, establishes it as a valuable resource for advancing the field of drone detection and counter-measure development.

A mixed deep learning approach for planning UAV routes in changing conditions

Zhang [59] presented a paper with the use of Unmanned Aerial Vehicles (UAVs) in military operations has grown in importance. UAVs are typically operated in uncertain

environments during actual missions, necessitating careful planning to adjust in real time. For UAVs to navigate through challenging terrain, an efficient path-planning algorithm is also necessary. But for UAV path planning, the widely utilized Rapidly-exploring Random Tree (RRT) method frequently produces less-than-ideal routes that need a lot of post-processing to make them better. The 32-Neighbor A search algorithm, which expands the traditional 8-neighbor search to provide smoother path options, is illustrated in Figure 2.3. Its variations have higher memory and compute costs, but they can converge to the best answer. However, purely learning-based approaches could lead to poor planning or overly lengthy courses created in order to avoid obstacles. In this study, the authors provide a neural network architecture including CNN, LSTM, and an attention mechanism for UAV path planning that combines a variety of sensor inputs. The neural module and informed sampling techniques are then combined by the deep learning-enhanced RRT algorithm to provide adaptive and effective pathfinding in dynamic contexts. The authors method outperforms the state-of-the-art Layered RQN, Informed RRT*, and Batch Informed Trees (BIT*) algorithms in terms of overall performance after carefully weighing the trade-offs between path optimality, computing efficiency, and fewer turning angles.

Using multi-agent reinforcement learning for drone mapping Riccardo Zanol [60] presented a paper explaining drone mapping and surveying for objects of interest, such forest fires, landslides, or wild animals, has become more popular in recent years across a variety of academic communities. However, the usefulness of the drones is significantly limited by the requirement for a human pilot or a pre-planned flight path, particularly when an entire swarm is deployed. In order to demonstrate that the performance loss resulting from the absence of explicit optimization and pre-programmed knowledge of the system statistics is minimal in the swarm scenario, the researchers present a model of the drone survey problem and apply three well-known reinforcement learning techniques.

Dynamic Reward-Enhanced Q-Learning for Mobile Robot Path Planning Gharbi [61] developed Dynamic Reward-enhanced Q-learning (DRQL) as a new method to guide mobile robots through unknown spaces while avoiding obstacles. The research combines Q-learning methods with adaptive reward systems to enhance the speed of convergence and

exploration performance in standard reinforcement learning systems. The DRQL system uses an advanced reward system which evaluates both target proximity and obstacle distance to deliver advanced feedback to the learning system beyond basic reward systems. The system uses a reward calculation system which adjusts its output based on the robot's current position relative to its target and its distance from surrounding obstacles. The reward system gives positive rewards to the agent when it approaches the target but it penalizes the agent when it moves away from the target or gets too close to obstacles. The dynamic reward system helps the agent learn better paths because it minimizes random exploration and leads to faster policy discovery. The system consists of three operational stages which start with exploration followed by combined exploration and exploitation[62] before ending with final exploitation to solve complex path planning problems. The research team performed extensive simulations using three different environment setups which included different obstacle densities. The results showed that DRQL performed better than SRQL in all evaluation metrics because it reduced movement steps by 9.23-20.78% while maintaining superior reward accumulation during learning. The dynamic reward system enabled the robot to discover paths more efficiently because it achieved goal position faster in all tested environments. The research develops an improved learning system for mobile robots which uses reinforcement learning adaptability to benefit from dynamic reward-based strategic guidance. The research conducted its tests within simulated environments yet real-world testing remains essential to determine the system's practical implementation in physical robotic systems. The DRQL method demonstrates excellent potential for navigating through uncharted and partially known areas because it handles complex path planning problems effectively.

2.2 A-Star Algorithm for Drone System

UAV trajectory optimization using the A-star algorithm and clustering-based strategic 3D deployment for improved disaster response UAV trajectory optimization using the A-star algorithm and clustering-based strategic 3D deployment for improved disaster response Humairah Hamid [63] presented a broad spectrum of communication and information technologies is currently being investigated for their potential applications in

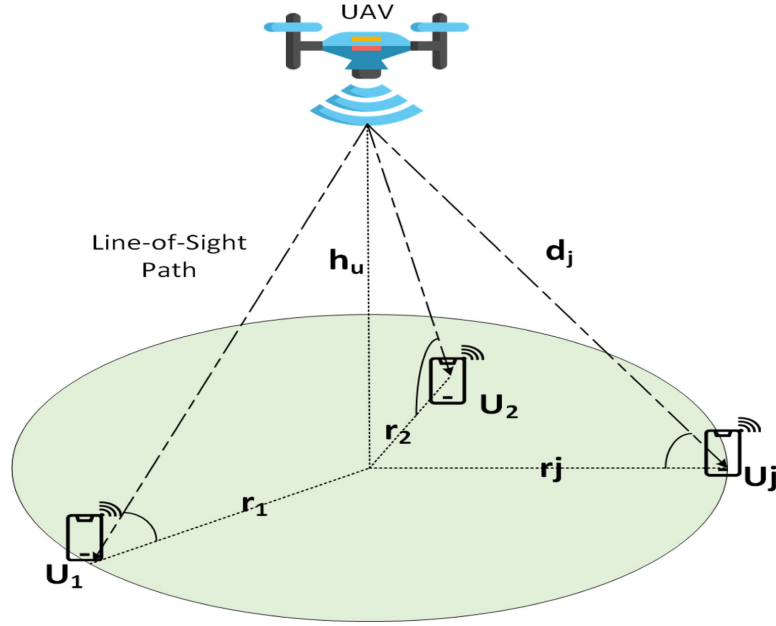


Figure 2.4 System model [63]

disaster management. A high level of situational awareness, combined with a prompt and accurate response, is essential for the preservation of life during catastrophe scenarios. This study presents a novel communication strategy employing Unmanned Aerial Vehicles (UAVs) as aerial base stations for providing connectivity to the affected area. The system model showing the UAV deployment architecture with line-of-sight paths and user connectivity is depicted in Figure 2.4. The system takes advantage of the flexibility and quick deployment characteristics of UAVs. The main focus is to determine the optimal UAV deployment along with trajectory planning to ensure connectivity in areas where conventional base stations are inaccessible. The proposed system employs two types of UAVs: cluster UAVs which act as stationary base stations and relay UAVs acting as mobile base stations. A three-step strategy is proposed to find the suitable location of cluster UAVs, optimize their height and power, and find the optimal trajectory of the relay UAVs to maximize the percentage of users served. Gaussian Mixture Model (GMM) clustering is employed to determine the optimal horizontal location of cluster UAVs. An optimization problem is framed for finding out the optimal height and power for cluster UAVs. Heuristic-based A-star algorithm is used to find out the trajectory of the relay UAVs which can efficiently minimize the overall path length while avoiding obstacles. The simulation results confirm the effectiveness of the proposed approach and demonstrate performance enhancement by comparing it with the benchmark schemes.

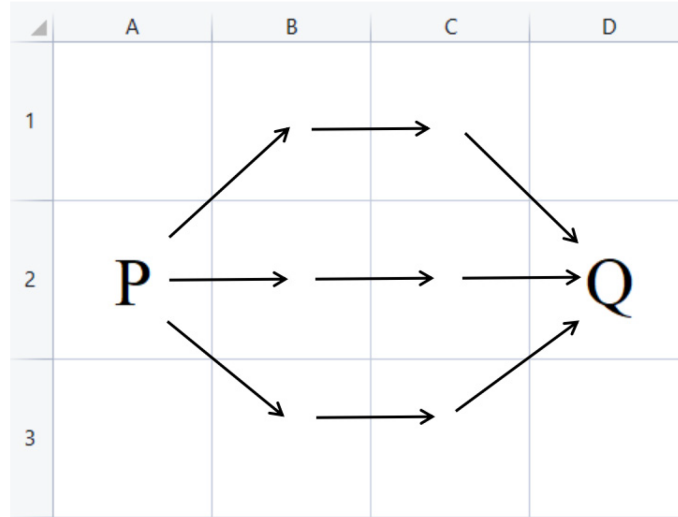


Figure 2.5 Path selection during search [64]

Improved method using A-Star algorithm for model R5DOS Based on the R5DOS model, [64] this paper suggests a three-dimensional space UAV path planning model to address the issues of the A-star algorithm's lengthy computation times and massive computation volumes in three-dimensional space. Together, the enhanced A-star algorithm and the improved R5DOS intersection model are used. They build a local search process together, and by lowering the number of search nodes, the R5DOS path planning model is established. The path selection mechanism during search, demonstrating how the algorithm evaluates multiple route options from point P to point Q, is shown in Figure 2.5. By simulating the path planning model with MATLAB software, the model can significantly lower the computational complexity and number of nodes of the A-star algorithm in three-dimensional spaces, as well as the UAV's calculation time. Lastly, the authors contrast the refinement of the A-star algorithm with both the original and geometric versions. It is demonstrated by the final fitting result that the enhanced A-star algorithm requires fewer node visits and has a shorter computation time. In general, the simulation results validate the efficacy of the enhanced A-star algorithm and can serve as a guide for further path planning algorithm research.

Optimal route planning for stealth unmanned aerial vehicles in a 3D network radar environment using a modified version of the A-Star algorithm Zhe Zhang [65], presenter a paper on Path security and search efficiency of penetration paths are the two most crucial aspects of mission execution for stealth unmanned aerial vehicles (UAVs). The

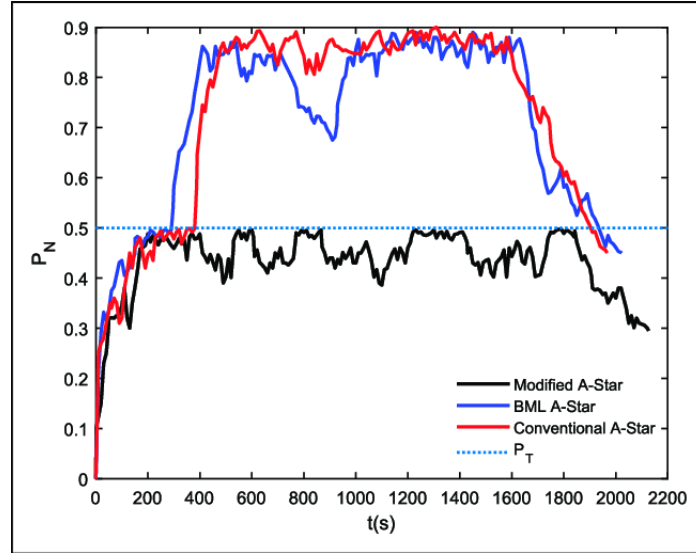
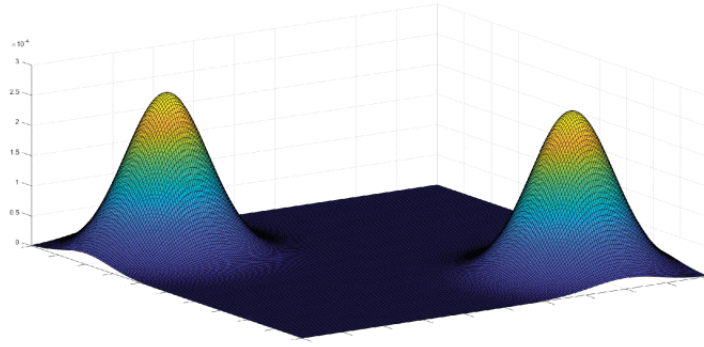


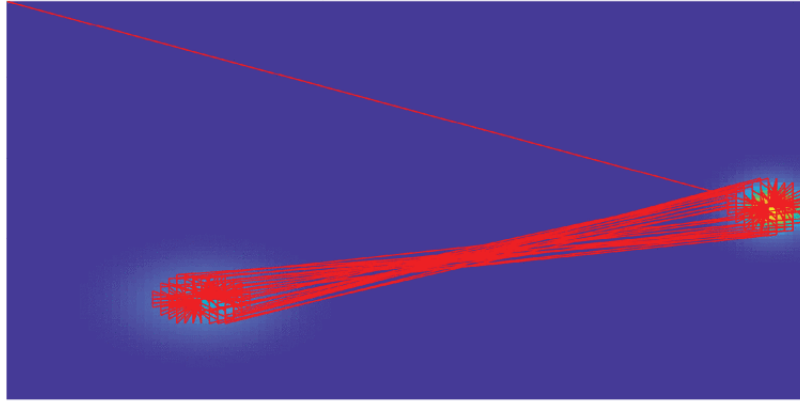
Figure 2.6 Radar detection probability of different algorithms in the third threat scenario [65]

optimal penetration path planning technique that takes into account the network radar system, the dynamic radar cross-section of stealth UAVs, and kinematics principles is examined in this article. The traditional A-Star algorithm was modified to include a 3D bidirectional sector multilayer variable step search strategy and the radar threat estimation function. The radar detection probability comparison among different algorithms in the third threat scenario is presented in Figure 2.6, demonstrating that the modified A-Star algorithm maintains lower detection probability compared to conventional and BML A-Star approaches. This modified A-Star algorithm seeks to meet both the algorithm searching efficiency and waypoint accuracy requirements. Attitude angle constraints and the radar system's rank-K fusion criterion were then met by the simultaneous selection of new waypoints using the suggested penetration path planning method. The conventional A-Star algorithm, bidirectional multilayer A-Star algorithm, and modified A-Star algorithm were also used for comparative analysis of various algorithms in order to resolve the penetration path issue that UAVs encounter under different threat scenarios. In a 3D complex network radar environment, the simulation results demonstrate the efficacy of the suggested path planning scheme by showing that the paths produced by using the modified algorithm have optimal path costs and higher safety.

The Implementation of A* Algorithm for Global Path Planning in Autonomous The research by Arfinati et al. [66] demonstrated A* algorithm implementation for autonomous ground vehicle path planning through complex road environments with moving obstacles. The



(a) Probability distribution of the target's location.



(b) UAV's path for first 300 visits.

Figure 2.7 A search and detection autonomous drone system: from design to implementation [67]

researchers used a kinematic vehicle model in their MATLAB simulation to test the algorithm on an 800-meter curved road with four moving obstacle vehicles. The A* algorithm operated within a binary occupancy grid mapping system which used 0 to indicate free spaces and 1 to show occupied areas. The authors obtained an average total estimated cost value of 2475 while the vehicle moved 139 meters during 10 seconds at a fixed speed of 15 m/s. The research proved A*'s ability to create optimal paths which both minimize distance and ensure safety in changing environments, but the study focused on ground vehicle kinematics and fixed road layouts instead of drone-specific aerial navigation flexibility.

2.3 Search and Detection Operation in Drone System

Path planning algorithm based on Bayesian Inference Path planning algorithm based on Bayesian Inference M. Khosravi et al. [67] presented a comprehensive autonomous drone system for search and detection operations, specifically covering search and rescue (SAR) and wildfire detection scenarios. Their approach integrates two key components: a

path-planning algorithm based on Bayesian inference and a target detection system using a residual neural network (ResNet). The system's probability distribution of the target's location and the resulting UAV path for the first 300 visits is illustrated in Figure 2.7, showing how the drone prioritizes high-probability areas during search operations. The path planning algorithm optimizes the drone's route by utilizing probabilistic knowledge of target locations, while the ResNet-based detection system achieves more than 91% accuracy in identifying targets from aerial imagery. The system employs a sliding window technique to manage computational complexity and incorporates probability updating through Bayesian inference for improved search efficiency. Their system demonstrated superior performance through simulation and real-world experiments compared to benchmark algorithms, achieving significantly reduced search times and energy consumption. Their approach proved five times faster than traditional zigzag search patterns for a Gaussian mixture distribution scenario. The authors evaluated their system by using a DJI Mavic 2 Zoom drone equipped with a 4K camera, testing it in various conditions involving fire detection and person detection in snowy environments. While proving effective in these different scenarios, the authors note potential improvements through ensemble ML models and extension to multi-UAV operations.

SAR Operations in disaster environment with bio radars Mittal et al. [68] presented a vision-based autonomous UAV navigation and landing system designed explicitly for Urban Search and Rescue (USAR) operations in post-disaster environments. The system enables UAVs with bio radars to autonomously navigate through and land on collapsed buildings to locate survivors. Their key contributions include (1) A novel landing site detection algorithm that evaluates multiple hazard factors, including terrain flatness, steepness, depth accuracy, and energy consumption through weighted cost maps; (2) An efficient exploration strategy using a next-best-view planner combined with TSDF-based 3D reconstruction for mapping disaster environments; and (3) A minimum-jerk trajectory generator for safe landing maneuvers. The system ran in real time on an embedded computer. It was extensively validated both in simulation using their introduced synthetic dataset of 1.2 million images and in real-world experiments at a rescue training facility. Results demonstrated reliable detection of safe landing sites on rubble piles and debris, outperforming existing approaches that tend to Favor flat

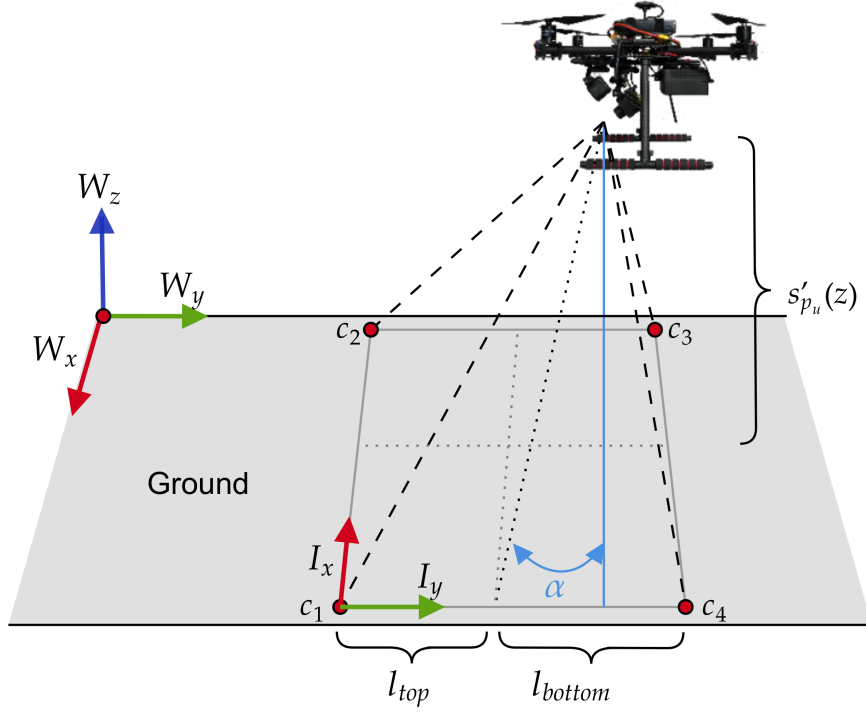


Figure 2.8 Field of view (FOV) projection and footprint extent of a vision-based sensor.[69]

ground surfaces. The authors also released the first large-scale dataset of collapsed buildings with ground truth depth, surface normal, and semantic labels to facilitate further research.

Autonomous Motion Planning System for Outdoor Environments with Uncertainty in Object Detection Using Drones Sandino et al. [69] presented a modular system for autonomous motion planning and object finding using small UAVs, focusing on search and rescue (SAR) missions. The system employs a Partially Observable Markov Decision Process (POMDP) to enhance UAV autonomy for obstacle avoidance, environment exploration, and object inspection under uncertainty conditions. The field of view (FOV) projection and footprint extent of the vision-based sensor is illustrated in Figure 2.8, showing how the camera setup on the UAV frame defines the pointing angle and determines the coordinates of the footprint corners. The architecture integrates Software-in-the-Loop (SIL) for PX4 autopilot emulation and Hardware-in-the-Loop (HIL) for onboard inference using a MobileNet SSD detector and motion policy computation via TAPIR and ABT solver. The system was evaluated in simulation and preliminary real flights using three flight modes: mission (baseline), offboard (POMDP-based), and hybrid (combined approach). Testing involved locating a victim in two scenarios with varying occlusion levels. Results showed offboard and hybrid modes achieved

superior detection accuracy (100% and 92.5% respectively) compared to the baseline, with minimal collision rates (1.25% overall) during navigation. The research notably demonstrated improved robustness against false positives and the ability to handle object detection uncertainty in outdoor environments using resource-constrained hardware, though flight times increased due to the comprehensive inspection approach. The study recommends future work including detailed real-flight analysis, evaluation with additional sensors like thermal cameras, and investigation of more complex scenarios with multiple or dynamic targets.

Chapter 3 Path Planning with Hybrid A* and Reinforcement Learning

3.1 System Overview

The system for path planning unites various elements which fulfill three essential requirements for drone autonomy in real-time operations: it performs immediate planning while detecting moving obstacles and it creates routes based on battery life for power savings and it adjusts its route for navigating through difficult urban areas. The system presents an extensive solution which unites traditional pathfinding methods with contemporary reinforcement learning approaches. The system architecture implements defense-in-depth design principles through its multiple algorithmic layers which ensure both reliability and redundancy. The system uses multiple components which work together to prevent complete system failure when one component stops working or performs below expectations. The system uses A* pathfinding as a backup when the reinforcement learning component generates decisions with low confidence because A* pathfinding guarantees optimal path results. The system design maintains operational success when system operations encounter unexpected situations which training data failed to show. The separation of concerns principle guides component design through a modular structure which assigns specific duties to each component. The system uses MR-QLearning to learn and adapt which creates intelligent navigation policies through its experience-based learning process. The A* algorithm provides optimal solution assurance because it generates correct and efficient paths during learning-based operations which produce unpredictable results. The hybrid layer functions as a control system which determines when to use learned policies and when to apply traditional methods. The system design includes separate modules which allow developers to test and validate individual components while preserving the functionality of other system elements. As illustrated in Figure 3.1, the system operates through four sequential phases that collectively enable intelligent autonomous navigation. Each phase builds upon the outputs of preceding phases while maintaining the flexibility to adapt to changing operational requirements.

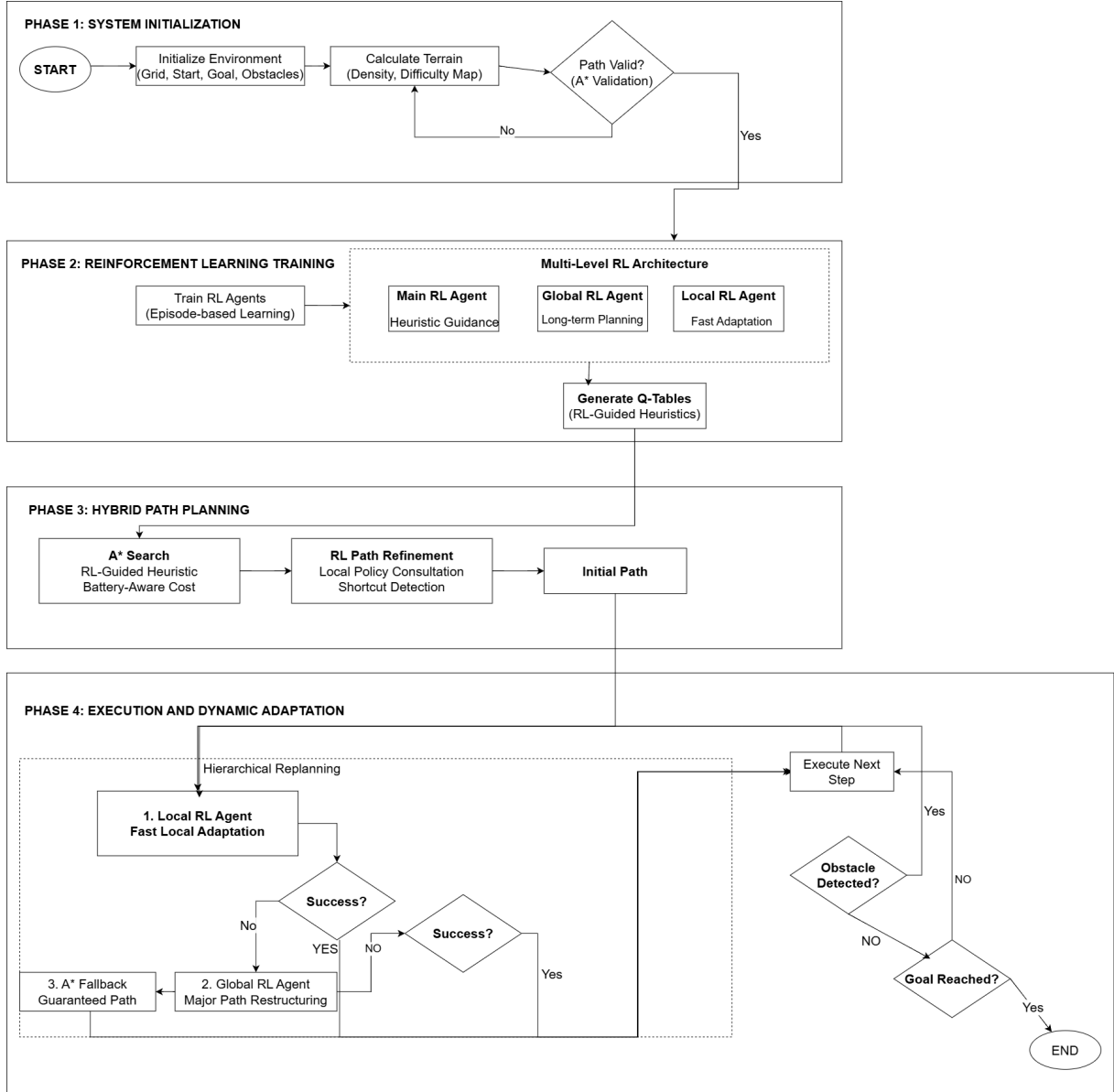


Figure 3.1 Comprehensive flowchart of the Hybrid A* + Reinforcement Learning system illustrating the four operational phases

3.2 Static Global Path Planner Based on A* Algorithm

The A* search algorithm serves as the deterministic basis which the Hybrid system uses to develop its intelligent navigation system. The algorithm solves the energy limitations of self-operating drones through its implementation of battery-conscious optimization methods and its calculation of step-related expenses. The algorithm benefits from experiential knowledge through RL-guided heuristics which enable reinforcement learning integration while preserving theoretical optimality guarantees. The hybrid system depends on A* as its dependable backup

system which maintains navigation stability through uncertain learned policies to create a necessary safety feature for search and rescue operations in real-world environments.

3.2.1 Heuristic Function Design

The A* implementation uses the Manhattan distance as its base heuristic function which calculates the total absolute difference between the current node position and the goal position through horizontal and vertical distance calculations. The Manhattan distance between a node located at (x, y) and a goal position at (g_x, g_y) equals the sum of the absolute differences between x and g_x and between y and g_y . The heuristic functions as an admissible and consistent solution for grid-based environments which allow four-directional movement because it calculates the Manhattan distance which represents the shortest path to the goal when there are no obstacles. The mathematical properties of this heuristic ensure that A* will always find the shortest path when one exists. The Manhattan distance serves as an effective base heuristic but the Hybrid system builds upon this foundation through context-based adjustments which modify the heuristic according to operational requirements. The following subsections explain how these extensions convert the fixed heuristic into an adaptive evaluation system which adapts to battery limitations and acquired environmental data.

3.2.2 Battery-Aware Heuristic Enhancements

The A* implementation includes a battery-aware heuristic enhancement which adjusts path evaluation through the drone's current energy levels. The system fulfills the essential operational need for drones to arrive at their targets before their batteries run out while the need for arrival becomes more critical as the drone's energy supply decreases. The battery-aware heuristic uses multiplicative scaling factors which apply to the base Manhattan distance through discrete battery threshold values. The heuristic functions as usual when battery capacity stays above fifty percent because it enables the algorithm to select paths based solely on distance without any modifications. The search direction shifts toward shorter paths when battery levels reach below fifty percent through the application of a 1.3 scaling factor to the heuristic. The system increases its scaling factor to 2.0 when battery capacity reaches thirty percent or lower to select the most efficient path toward the objective. The system implements a

progressive response system which makes its operations more restrictive when energy resources become scarce because battery power decreases. The battery-aware enhancement preserves heuristic admissibility through cost increases which never reduce the estimated values. The base Manhattan distance function already shows the lowest possible cost so any multiplicative factor between 1 and above will maintain the guarantee that the heuristic provides accurate path cost estimates. The property preserves optimal solutions when the cost function includes battery factors while the search system adjusts its behavior to handle power limitations.

3.3 Adaptive Navigator Via MR-Qlearning (Modified Reward)

The workflow of MR-QLearning algorithm appears 3.2 the algorithm starts by setting up the Q-table and Replay Buffer and Confidence Threshold and Q-value History while it performs an optional Transfer Learning check to use pre-trained knowledge from similar environments. The agent performs ϵ -greedy action selection in each episode before it takes actions to receive rewards and then it applies the Bellman equation to update its Q-values. The system performs two independent operations during update procedures because the Experience Replay Buffer maintains experience data for batch learning and Uncertainty Quantification monitors Q-value variance to determine confidence levels. The system uses Experience Replay to select additional learning batches after finishing each episode while the Adaptive Confidence Threshold determines the RL-A* switching point through success rate evaluation. The integrated system structure solves Q-learning problems through its ability to enhance data usage efficiency and make decisions based on confidence levels and perform automatic parameter tuning and environment knowledge sharing.

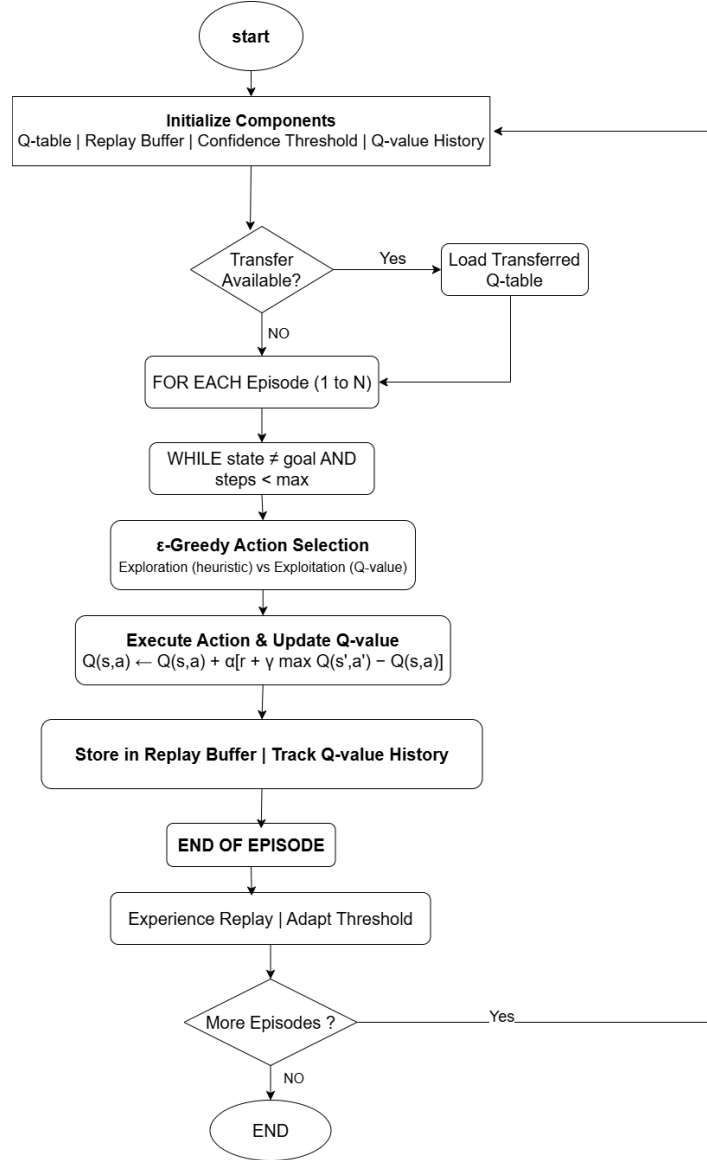


Figure 3.2 Mr-QLearning Algorithm Flowchart

3.3.1 Base Q-Learning Framework

The base Q-learning framework serves as the foundation for MR-QLearning which requires its own description. The fundamental knowledge base enables the understanding of how each addition builds upon the original algorithm while solving its built-in problems.

3.3.1.1 State Space Design

The discrete grid-based environment of MR-QLearning uses a state space which includes all grid positions because each state in the system corresponds to a particular cell where the drone can be found. The discrete grid representation served multiple essential purposes during the selection process. The system enables tabular Q-learning to discover optimal

policies through its manageable state space which operates without needing neural networks or function approximation methods thus maintaining algorithm simplicity and interpretability and operational efficiency. Second grid-based representations match urban environments because drones operate between specific waypoints and buildings and other distinct landmarks. The discrete formulation allows to conduct formal evaluations of algorithm properties which include convergence proof and optimality verification for establishing the theoretical basis of this method.

3.3.1.2 Action Space Design

The system features an action space which allows the drone to move their character in four basic directions which include up and down and left and right. The drone will move to the next cell in the chosen direction when each action is performed unless the target cell contains an obstacle. The four-direction action space provides both basic functionality and detailed control options. The eight-directional movement system which includes diagonal directions offers better flexibility but the four-direction system limits action space complexity to half of the original size while it shortens learning time by reducing Q-value learning requirements and makes collision detection operations more straightforward. The drone navigation remains functional because smooth flight paths can be generated from waypoint sequences during real-time mission operations.

3.3.1.3 Reward Function Design

The reward function exists to direct the agent toward its objective while preventing it from performing unwanted actions. The reward structure consists of multiple elements which deliver precise learning indications during the entire training period. The agent receives a substantial positive reward when it achieves the goal state which provides powerful positive feedback for finishing missions and makes goal achievement the most valuable action for the agent. The system imposes a major penalty when the agent tries to enter an obstacle or leave the environment area because this penalty stops dangerous movements and helps the agent learn to identify and stay clear of obstacles. The progress bonus system which rewards agents based on their distance progress toward the goal helps them move correctly toward the target while the

shaped reward component speeds up learning because it delivers instant feedback about their navigation advancement instead of requiring them to wait for the infrequent goal reward. The system implements a small penalty for each movement the agent makes to force the agent to choose shorter paths which results in better navigation efficiency.

3.3.1.4 Battery-Aware Reward Function Enhancement

The battery factor $\beta(battery)$ implements progressive urgency based on the remaining energy:

$$\beta(battery) = \begin{cases} 1.0 & \text{if } battery > 50\% \\ 1.3 & \text{if } 30\% < battery \leq 50\% \\ 2.0 & \text{if } battery \leq 30\% \end{cases} \quad (3.1)$$

The RL reward function includes battery awareness to teach the agent about navigating while minimizing energy consumption. The reward function includes a battery penalty system which becomes more severe when the battery charge drops. The base step penalty directly relates to battery usage during each step because it establishes a direct relationship between reward value and the energy expenses of each step. The reward system tracks battery usage because each step requires battery power which determines the reward value. The agent should only receive this base penalty when battery levels are high because it enables the agent to work on additional objectives. The system imposes additional penalties when battery levels reach warning thresholds. The system imposes a moderate penalty which increases based on the battery level distance from its threshold when the battery reaches medium capacity. The system imposes an increased penalty which grows stronger when battery power reaches its lowest point. The progressive penalty system generates rising levels of pressure which forces the agent to achieve its goal rapidly because the battery power decreases. The final reward includes the step penalty together with the battery penalty and the progress bonus. The RL agent learns to manage its goal pursuit activities while maintaining power efficiency through this system. The agent develops new behaviors through which it chooses shorter routes when its battery runs out but explores more when its battery level is sufficient.

3.3.1.5 Q-Value Update Mechanism

The Q-value update uses the standard Bellman equation to calculate new Q-values through a process which combines the previous value with the current reward and the future value that has been discounted. The learning rate parameter determines the speed at which new information replaces existing knowledge while the discount factor shows the agent's preference between present and future rewards. The selected parameters emerged from experimental evaluation to achieve an optimal learning pace which maintained system stability while the agent acquired knowledge without losing previously acquired knowledge. The update mechanism performs multiple iterations to improve value estimates until they reach their optimal action-value function which produces the best navigation behavior through greedy policy selection.

3.3.2 Experience Replay Buffer

experience replay adapted for heuristic-guided tabular Q-learning in pathfinding tasks, extending techniques previously applied in deep RL contexts[70]. The research solves essential problems which occur when Q-learning operates with standard training data processing methods and environmental interaction value extraction methods. The agent in standard Q-learning learns from sequential experiences which it uses for a single Q-value update before moving to the next experience. The sequential learning method produces multiple issues which restrict both learning performance and system stability. The first factor shows high correlation between consecutive events because these events stem from the sequential process of the same trajectory. The agent experiences three consecutive rightward movements which produce three experiences that stem from the same rightward direction. The independent and identically distributed assumption which learning algorithms need for operation becomes violated when these events occur in a specific order which leads to unstable or slow learning processes. The experience replay mechanism operates through a structured storage and sampling process. The training process involves storing each experience tuple which contains the current state and action and reward and next state into a replay buffer. The system maintains a maximum storage capacity for its buffer which removes the oldest entries when it reaches its maximum capacity to

accommodate incoming new data through first-in-first-out replacement. The system maintains a buffer which stores recent experiences from the past while keeping enough diverse information to enable learning success.

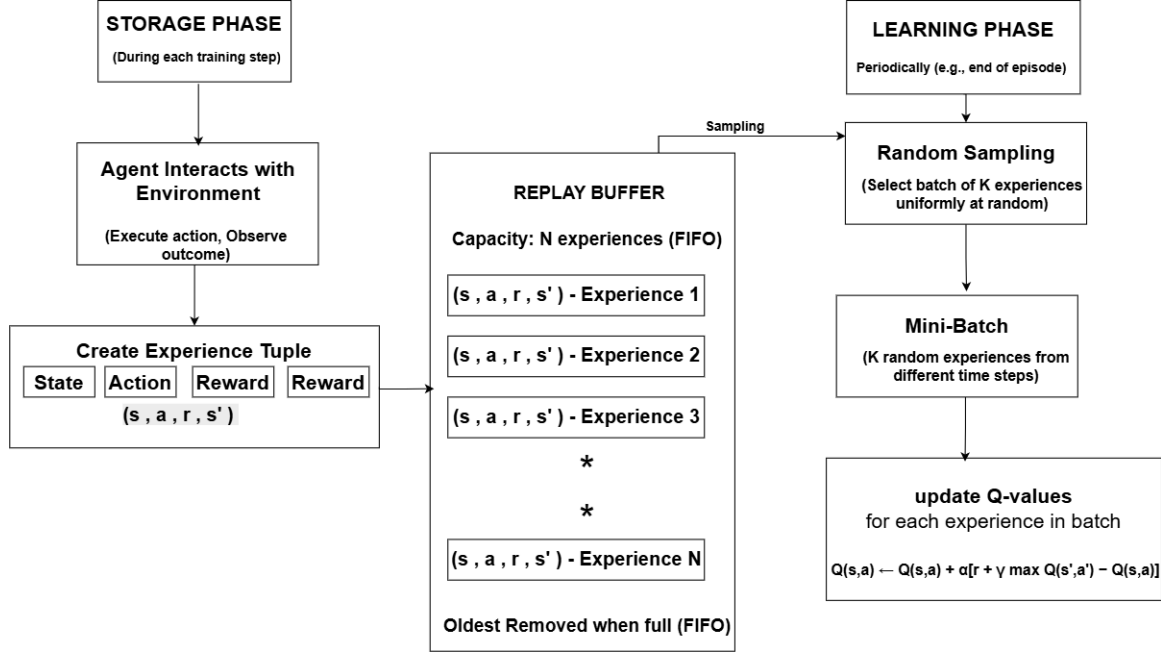


Figure 3.3 Experience Replay Buffer Mechanism

As presented in 3.3, the training process includes occasional replay phases which select random experiences from the buffer through uniform sampling. The system runs Q-value updates for every sampled experience as if these experiences happened at the moment they occurred. The technique requires random sampling because this method selects experiences by chance instead of following their natural sequence which helps to eliminate the time-based connections found in the original data. The agent learns from different experiences which are independent from each other which produces better learning stability and efficiency. The replay frequency together with batch size serve as essential parameters which determine both the learning outcomes and the amount of computational work required. The system becomes slower when it runs the replay function too many times because it uses its resources to process new data instead of getting new experience. The technique becomes less effective when the system does not play back experiences at appropriate intervals. The batch size determines the number of experiences which get replayed during each replay phase because bigger batches deliver better learning results but they need additional processing power. The system maintains these parameters at levels which deliver substantial performance gains while keeping the

computational requirements at a reasonable point. The authors selected these particular values through experimental evaluation of the system on standard navigation tasks. The experience replay mechanism delivers multiple essential advantages which work together to enhance learning performance. The system achieves better sample efficiency because each experience leads to multiple Q-value updates instead of a single update which enables the system to learn more from the same environmental interactions. The process of decorrelated learning occurs through random sampling which removes the time-based connections between consecutive experiences to produce stable learning patterns that achieve faster convergence. The system maintains essential experiences because it saves vital moments like goal achievement and obstacles which users can replay multiple times for complete learning of these critical events. The agent experiences reduced forgetting because it revisits previous experiences at intervals which decreases its probability of losing learned information while it explores new parts of the state space thus preserving its policy performance throughout the environment.

3.3.3 Uncertainty Quantification

Uncertainty quantification generates multiple uncertainty factors to support decision processes based on confidence levels. The system needs to determine its trust level for learned Q-values because this question determines the success of hybrid reinforcement learning systems with classical search systems. The uncertainty quantification mechanism solves this problem through its ability to monitor Q-value changes during updates while generating a confidence score based on multiple factors. The system uses multiple learning quality indicators instead of visit count as a single metric to identify unstable states which results in better confidence estimation accuracy. The main finding shows that Q-value learning confidence emerges from three essential elements which include state exploration completion and learning stability and the presence of beneficial actions in Q-value estimates. The system will identify authentic reliable states through its combination of these elements which also detect states that seem reliable based on basic measurements yet remain uncertain.

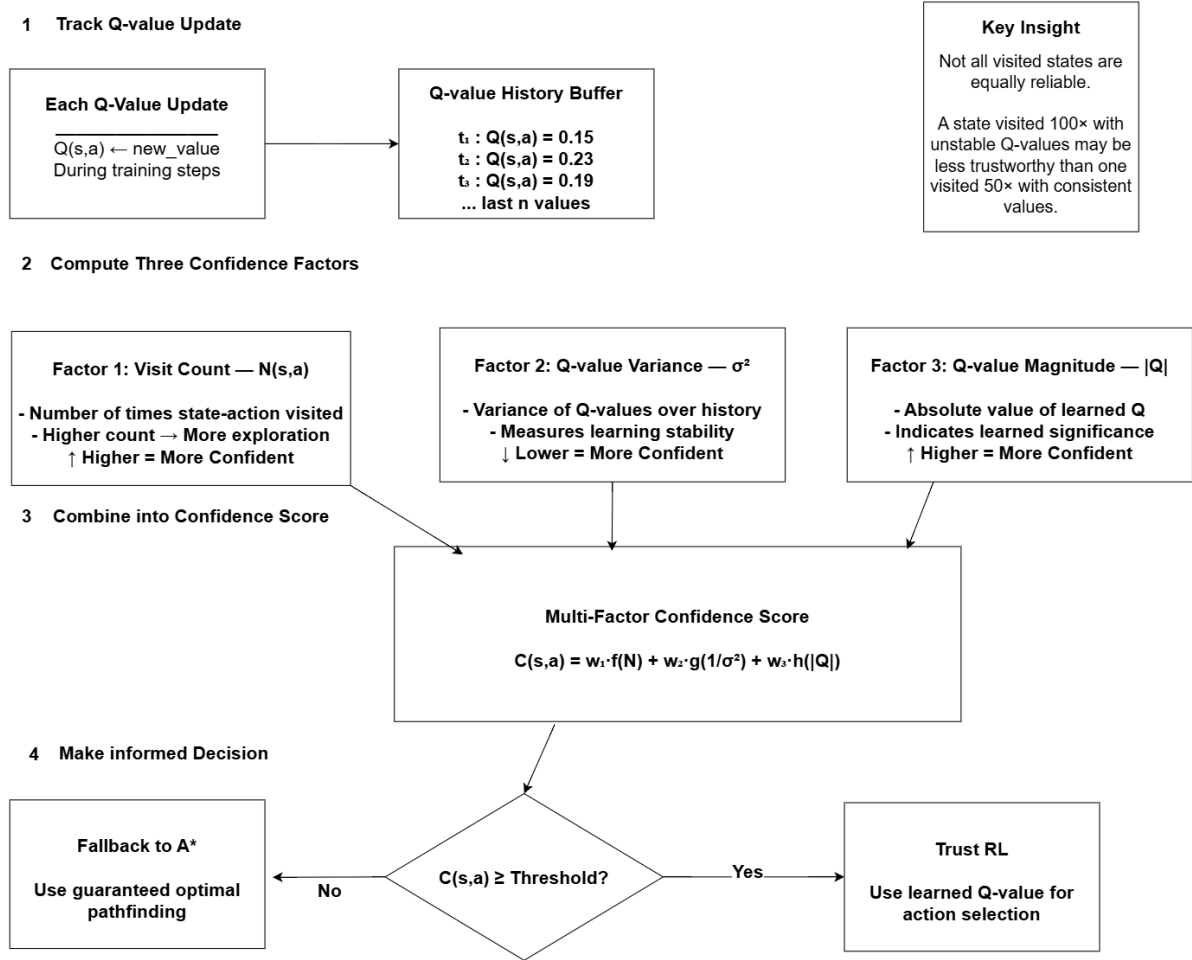


Figure 3.4 Uncertainty Quantification Mechanism Chart

As presented in 3.4, the operational mechanism operates through a system which stores Q-value information for all possible state-action combinations. The system adds new Q-value updates to the state-action pair history whenever it performs an update. The system maintains a limited history which contains only the latest updates because this approach enables the confidence measure to show the current learning status instead of using outdated information. The system uses historical data to generate three component scores which it combines into a final confidence rating. The visit count score evaluates exploration completeness through its calculation of state-action pair exploration depth which gets normalized to a standard range for combination with other factors. The system assigns low scores to states which receive few visitors but awards high scores to states which attract numerous visitors until the visit count reaches its maximum capacity. The variance score evaluates learning stability through Q-value history variance calculation which shows stable learning when Q-values remain consistent but becomes unstable when their values change significantly. The system uses inverted and

normalized variance to generate confidence scores which increase when the variance becomes lower. The magnitude score evaluates policy quality through Q-value magnitude assessment which shows positive Q-values for beneficial actions that generate rewards and negative Q-values for actions that produce negative outcomes. The Q-values of positive states lead to higher confidence scores because these states contain learned advantageous actions. The final confidence score results from a weighted calculation that combines these three components. The weights indicate the relative significance of each factor because visit count holds the highest value because it represents the core exploration metric yet variance and magnitude share equal lower values as additional measurements to enhance the basic indicator. The weighted combination method produces an overall uncertainty measurement which includes various elements that determine learning quality. The research presents the initial multi-factor uncertainty-aware confidence measure which operates between RL-A* hybrid pathfinding systems. The system achieves better decision-making about policy trust through improved confidence estimation which results from using statistical learning quality metrics instead of basic visit number tracking. The hybrid navigation system achieves enhanced robustness and reliability through this approach.

3.3.4 Adaptive Confidence Threshold

The adaptive confidence threshold represents the third improvement element of MR-QLearning because it enables automatic A* and RL switching point optimization without needing user-defined parameters. The hybrid RL-A* system uses a confidence threshold to decide when it should transition from executing learned RL policies to running the guaranteed A* algorithm. The system uses A* when state confidence scores drop below the established threshold but it uses RL when these scores exceed the threshold. The selection of this threshold value requires special attention because it determines when the system will activate RL: high values result in infrequent RL activation which prevents the system from adapting while low values cause the system to execute RL-based policies that produce unreliable results. The adaptive threshold mechanism tracks success rates from previous training sessions to determine its operation. The system tracks episode results through success or failure metrics

which it uses to calculate average success rates during scheduled intervals. The threshold adjustment logic runs at scheduled points which occur throughout the training process. The system evaluates performance through two established thresholds which include a minimum success requirement and a maximum success standard. The system reduces the confidence threshold by a small amount when the average success rate drops below its lower performance threshold. The system depends on A* for reliable pathfinding because of this development. The system raises the confidence threshold by the adjustment rate when the average success rate surpasses the upper threshold which shows excellent performance. The system becomes able to trust RL decisions through this implementation. The system sets boundaries for the threshold which should be within a normal range to stop the entry of abnormal values. The system must provide RL with at least one chance to contribute through its operations while preventing RL from depending too heavily on its performance even when it shows success. The system maintains stability through this limited adjustment process which enables significant changes.

3.3.5 Transfer Learning Framework

Transfer learning framework allows the system to apply learned knowledge between environments that share similarities to achieve faster training times and better starting performance. The transfer learning framework consists of three separate operational stages:

1. Q-Table Export: The Q-table which the system learned during training in the source environment becomes available for export. The Q-table shows the agent's learned policy through its accumulated knowledge about which actions lead to beneficial outcomes in different states. The export process enables users to create a Q-table duplicate which becomes available for future application to different operational settings.
2. The system performs a compatibility check to verify that the source and target environments share enough common characteristics before starting knowledge transfer. The state overlap ratio between environments is calculated through a proportion that shows which states appear in both environments. The system performs knowledge transfer when the overlapping information between source and target domains reaches or

surpasses a predetermined similarity threshold. The system will reject transfer operations when the threshold is not met because this prevents negative transfer which occurs when inappropriate knowledge reduces system performance.

3. Knowledge Initialization: The Q-values from the source become the starting values for the Q-table of the target environment after confirming compatibility. The source provides Q-values to states which appear in both environments but the target states with exclusive presence receive their values from default settings. The agent proceeds with its training in the target environment to develop its transferred knowledge while it discovers new states which exist only in the target environment.

3.3.6 Mathematical Formulations

The MR-QLearning system depends on multiple mathematical models which determine its ability to learn and make decisions. The section contains fundamental equations which describe system operation and serves as the theoretical basis for system execution.

1. Q-value update equation: The Q-value update follows the standard Bellman equation which is expressed as $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$, where $Q(s, a)$ represents the action-value function for state s and action a . The learning rate α is set to 0.7 which determines the size of updates and the speed at which new information replaces current knowledge. The discount factor γ receives a value of 0.9 which determines how much future rewards count against current rewards. The variable r shows the direct reward which becomes available after performing action a and s' represents the state which follows the action. The agent uses this formulation to perform temporal difference learning which enables it to improve its value estimates at each step until it reaches the optimal action-value functions.
2. Multi-Factor Confidence Score: The uncertainty quantification system generates confidence scores by combining three factors through weighted calculation which produces $Confidence(s, a) = w_1 \cdot VisitScore(s, a) + w_2 \cdot VarianceScore(s, a) + w_3 \cdot MagnitudeScore(s, a)$. The system calculates $VisitScore(s, a)$ through the formula $VisitScore(s, a) = \min(visit_count(s, a)/10, 1.0)$ which transforms exploration depth

into a value between 0 and 1 according to the number of state-action pair visits. The variance score function $VarianceScore(s, a) = \max(0, 1.0 - \min(\sigma^2(Q_{history})/100, 1.0))$ assesses learning stability by evaluating Q-value history variance where lower variance values indicate higher confidence. The MagnitudeScore function calculates its value through $MagnitudeScore(s, a) = \max(0, \min(\max(Q(s, \cdot))/100, 1.0))$ which assesses policy quality by maximizing Q-values between 0 and 1. The function produces positive values when the learned actions lead to beneficial outcomes. The system defines weight parameters which include $w_1 = 0.4$ for visit count to measure exploration completeness and $w_2 = 0.3$ for variance score to measure learning stability and $w_3 = 0.3$ for magnitude score to measure policy quality.

3. Adaptive Threshold Adjustment: The adaptive confidence threshold adjusts its value based on performance history through conditional rules. The system uses A* pathfinding as its primary method when the average success rate drops below 0.7 because the threshold value decreases through the formula $threshold \leftarrow \max(3, threshold - adjustment_rate)$. The system increases its trust in RL decisions through $threshold \leftarrow \min(10, threshold + adjustment_rate)$ when the average success rate reaches 0.9. The adjustment rate has been set to 0.1 while the threshold range has been restricted between 3 and 10 to stop the generation of abnormal values. The system calculates success rates through a 10-episode window which helps maintain stable learning processes.
4. Transfer Learning Compatibility: The transfer learning framework determines environment compatibility through state overlap ratio calculations. The similarity measurement uses the formula $Similarity = |S_{source} \cap S_{target}| / \max(|S_{source}|, |S_{target}|)$ to calculate the value. The sets S_{source} and S_{target} contain all states from the source Q-table and the target environment respectively. The system allows knowledge transfer when Similarity reaches or exceeds 0.3 because it needs at least 30% state space overlap to start the transfer process.

3.3.7 Training Parameters

The complete set of training parameters which MR-QLearning used appears in Table 3.1. The values emerged from experimental testing which optimized the relationship between training speed and policy achievement.

Table 3.1 MR-QLearning Training Parameters

Parameter	Symbol	Value	Description
Learning Rate	α	0.7	Q-value update step size
Discount Factor	γ	0.9	Future reward weighting
Initial Epsilon	ε_0	0.25	Initial exploration rate
Epsilon Decay	δ_ε	0.995	Exploration decay per episode
Minimum Epsilon	ε_{min}	0.01	Minimum exploration rate
Training episodes	N	300	Maximum training episodes
Replay buffer size	B	1000	Maximum stored experiences
Replay Batch Size	b	32	Experiences per replay phase
Replay Frequency	f_r	5	Episode between replay phases
Q-Value History Length	H	20	Recent updates for variance calculation
Confidence Threshold	θ_c	5	Initial RL-A* switching point
Threshold Adjustment Rate	δ_θ	0.1	Threshold learning rate
Transfer Similarity Threshold	θ_t	0.3	Minimum overlap for transfer
Progress Reward Weight	w_p	5.0	Goal progress reward scaling
Collision Penalty	P_c	-50	Goal progress reward scaling
Goal Reward	R_g	200	Goal achievement reward
Early Stop Window	W_s	30	Episodes for convergence check
Early Stop Threshold	θ_s	0.9	Success rate for early termination

The learning rate of 0.7 allows the model to learn quickly at first while keeping the system stable and the discount factor of 0.9 helps the model focus on distant rewards which promotes goal achievement instead of short-term rewards. The epsilon-greedy exploration strategy starts with 25% exploration probability which decreases to 1% minimum to allow deep exploration at first while using learned policies for exploitation during later training stages. The experience replay buffer contains 1000 transitions which provide enough diversity for batch learning to be effective and the batch size of 32 strikes a balance between computational speed and learning performance. The confidence threshold system operates between 3 and 10 with a 0.1 adjustment rate to prevent performance oscillations while it adapts to changes in performance.

3.3.8 Convergence Analysis of MR-QLearning

The convergence properties of the MR-QLearning algorithm represent a critical theoretical foundation that distinguishes this approach from standard Q-learning implementations. This section presents a formal analysis of convergence guarantees, examining how the novel components affect learning stability and asymptotic behavior.

3.3.8.1 Theoretical Convergence Guarantees

The MR-QLearning algorithm inherits the fundamental convergence properties of tabular Q-learning while introducing modifications that affect the convergence rate. Under standard assumptions of infinite state-action visitation and appropriate learning rate decay, the algorithm converges to the optimal action-value function:

$$\lim_{t \rightarrow \infty} Q_t(s, a) = Q^*(s, a) \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A} \quad (3.2)$$

where Q_t represents the Q-value estimate at iteration t and Q^* denotes the optimal action-value function. The experience replay mechanism modifies the effective sample distribution but preserves convergence under the condition that all state-action pairs remain reachable through the replay sampling process.

3.3.8.2 Impact of Experience Replay on Convergence

The experience replay buffer introduces a form of off-policy learning that affects convergence dynamics. When experiences are sampled uniformly from the buffer, the effective learning becomes a weighted combination of temporal difference updates across the stored distribution. The convergence bound can be expressed as:

$$\mathbb{E}[\|Q_t - Q^*\|^2] \leq (1 - \alpha \cdot \gamma)^t \cdot \|Q_0 - Q^*\|^2 + O\left(\frac{\alpha \cdot \sigma^2}{B}\right) \quad (3.3)$$

where α represents the learning rate, γ denotes the discount factor, σ^2 indicates the variance in the replay buffer samples, and B represents the batch size. This formulation demonstrates that larger batch sizes reduce the variance term, leading to more stable convergence at the cost of increased computation per update.

3.3.8.3 Adaptive Threshold Stability Analysis

The adaptive confidence threshold mechanism introduces a feedback loop that could potentially destabilize learning if not properly bounded. The threshold adjustment rule ensures stability through the following invariant:

$$\theta_{\min} \leq \theta_t \leq \theta_{\max} \quad \forall t \quad (3.4)$$

The bounded nature of the threshold ensures that the system never completely abandons either the RL component or the A* fallback mechanism. Experimental observations indicate that the threshold typically stabilizes within a narrow range after approximately 50-80 training episodes, reflecting the system's adaptation to the specific environment characteristics.

Table 3.2 Convergence Metrics Under Different Configurations

Configuration	Episodes to 90 %	Final Success Rate	θ Stability Range	Variance
Base Q-Learning	185 ± 23	94.2%	N/A	0.089
+ Experience Replay	142 ± 18	96.8%	N/A	0.052
+ Uncertainty Quantification	128 ± 16	97.4%	[4.2, 6.8]	0.041

3.4 Path Optimization by Combining A* and MR-QLearning Approach

The hybrid method depends on the understanding that A* and RL systems bring different capabilities to the table. The A* algorithm provides guaranteed optimal path discovery when an optimal solution exists, but it depends on static heuristics and lacks the ability to acquire knowledge from past experiences. RL can learn from experience and adapt to patterns in the environment, but it requires training and may not converge to optimal policies. The system reaches optimal performance through A* optimization while using RL to adapt its operations.

3.4.1 RL-Guided Heuristic Learning

The RL-guided heuristic results from uniting various individual components. The base component uses the standard Manhattan distance heuristic which serves as the basic admissible estimation method. The heuristic maintains its connection to geometric distance between the current position and the goal through this approach. The RL adjustment component uses the highest Q-value which exists for the present state to generate its value. The RL agent learns to

avoid this state when the maximum Q-value becomes negative so the adjustment method boosts the heuristic value to make the state more expensive for A* exploration. The algorithm reduces the heuristic value when the maximum Q-value shows positive value because this indicates a beneficial state. The adjustment process needs to be controlled so it affects the base heuristic without taking over its operation. The system uses previously learned heuristic adjustments from successful searches together with a weight that the system learned. The system can update its learned heuristic values through successful search results which show the actual paths that users follow to achieve their goals. The last heuristic unites all its elements while it might need modifications based on both battery power and environmental challenges. This contribution represents the first RL-guided heuristic adaptation for A* search in pathfinding applications. The RL-guided heuristic uses various elements to build an adaptive evaluation function which follows this formula $h_{rl}(n) = (h_{manhattan}(n) + rl_adjustment(n) + h_{learned}(n) \times 0.3) \times \beta(battery) \times \tau(terrain)$. The base heuristic $h_{manhattan}(n)$ calculates the total absolute distance between x_n and x_{goal} and y_n and y_{goal} . The RL adjustment function uses $rl_adjustment(n) = -\max(Q(n, \cdot)) \times 0.1$ to calculate its value. The heuristic value increases when Q-values become negative because these values represent poor states but it decreases when Q-values become positive because these values represent good states. The 0.1 scaling factor maintains RL influence at a level which is half of the base heuristic strength. The learned heuristic component $h_{learned}(n)$ stores adjustments which result from successful path executions through a 0.05 scaling factor that prevents learned values from controlling the combined heuristic.

The battery factor β implements the battery-aware scaling which Section 3.2.2 describes and the terrain factor uses $\tau(terrain) = 1.0 + terrain_difficulty(n) \times 0.5$ to boost heuristic values by a maximum of 50% for cells located in challenging terrain zones. The A* optimality guarantee remains intact in the combined heuristic because it uses learned environmental knowledge. The base Manhattan distance serves as an admissible metric which RL and learned components use to provide adaptive guidance and battery and terrain factors use to set priorities based on operational constraints for context-sensitive path evaluation.

3.4.2 RL-Based Path Refinement

The A* generated path serves as the input for path refinement which checks all middle points except the starting and ending points. The system requests preferred actions from the Local RL agent at each step based on its current position. The system needs to determine if the RL agent's proposed action matches the path which A* generated. The reasonableness check evaluates two essential factors. The proposed solution needs to direct the search toward an accessible cell which does not have any barriers blocking its path. Second the distance between the RL-suggested cell and the RL-suggested cell needs to stay within a specific range which matches the distance between the A*-planned cell and its planned cell. The tolerance enables small variations because RL can approve steps which extend slightly beyond the original length when other factors make them more suitable. The evaluation process excludes all major deviations which would extend the timeline beyond acceptable limits. The RL suggestion will become the new A*-planned step in the refined path when it successfully passes both verification steps. The system operates through all intermediate stages while it adjusts various steps according to the preferences learned from RL. The final refined path keeps the A* optimal path structure but adds local adaptations which RL learned during its training.

3.4.3 Computational Complexity Analysis

Understanding the computational requirements of the hybrid system is essential for practical deployment in search and rescue operations. This section provides a comprehensive analysis of the time and space complexity for each system component.

3.4.3.1 Time Complexity Analysis

The overall time complexity of the hybrid system depends on the interplay between the A* search component and the reinforcement learning operations. For a grid of size $N \times N$ with obstacle density d , the following complexity bounds apply.

The A* search component exhibits a time complexity of $O(N^2 \log N)$ in the average case when using the Manhattan distance heuristic. This complexity arises from the priority queue operations required to maintain the open set during pathfinding.

The MR-QLearning training phase operates with complexity $O(E \times N^2 \times A)$, where E represents the number of training episodes, N^2 denotes the state space size for a grid environment, and A represents the action space cardinality (four directions in the implementation).

The hybrid path planning operation combines these components:

$$T_{\text{hybrid}} = T_{A^*} + T_{\text{refinement}} = O(N^2 \log N) + O(P \times A) \quad (3.5)$$

where P represents the path length and A denotes the action space size for the RL refinement queries.

3.4.3.2 Space Complexity Analysis

The memory requirements of the system are dominated by the Q-table storage and the experience replay buffer. The Q-table requires $O(S \times A)$ space in the worst case, though sparse representations can significantly reduce this for environments with limited reachable states.

The experience replay buffer maintains a fixed-size collection of experience tuples, requiring $O(B_{\max} \times 4)$ space where $B_{\max}$ represents the maximum buffer size and each tuple contains four elements (state, action, reward, next_state).

Table 3.3 Computational Complexity Summary

Component	Time Complexity	Space Complexity	Notes
A* Search	$O(N^2 \log N)$	$O(N^2)$	Priority queue
MR-QLearning Training	$O(E \times N^2 \times A)$	$O(S \times A)$	Q-table storage
Experience Replay	$O(B)$	$O(B_{\max} \times 4)$	Fixed buffer
Hybrid Planning	$O(N^2 \log N + P \times A)$	$O(N^2 + S \times A)$	Combined

3.4.4 Proposed Hybrid Approach Flowchart

The Hybrid A* + Reinforcement Learning system operates through the workflow 3.1 which demonstrates in detail. The flowchart shows the entire system execution process which starts with system initialization and ends with dynamic path adaptation for search and rescue scenarios through four separate phases.

Phase 1 System Initialization

The system starts by setting up the grid-based environment which includes both the fixed obstacles and the definition of both the starting point and the ending point. The system generates environment data before it starts calculating terrain characteristics which include obstacle density analysis and local difficulty assessments for each cell based on its surrounding cells. The system requires A* search for critical validation purposes to confirm that a path exists which allows movement between the starting point and the destination. The environment will keep regenerating until it produces a solvable configuration which enables all following operations to take place on accessible terrain.

Phase 2 Reinforcement Learning Training

The second phase requires the multi-level reinforcement learning architecture to learn through episode-based training. The three specialized RL agents received their training based on their specific positions in the system structure. The Main RL Agent learns to navigate through environments while producing Q-values which help improve A* search performance through RL-based heuristics. The Global RL Agent learns to create long-term plans because it needs to handle major path changes that occur when it meets substantial barriers. The Local RL Agent develops fast obstacle avoidance skills which enable it to adapt its tactics quickly. The MR-QLearning system operates with three new features which run simultaneously during training because it uses an experience replay buffer to enhance sample efficiency and uncertainty quantification for confidence assessment and adaptive confidence thresholds for making better fallback decisions. The system produces Q-tables which contain learned state-action values after training completion to support RL-based heuristic development in future stages.

Phase 3 Hybrid Path Planning

The system uses trained RL policies to generate its first path through the hybrid planning system. The A* search algorithm uses the RL-guided heuristic which combines the basic Manhattan distance with Q-value adjustments that the Main RL Agent learned during training. The system uses battery-aware cost modifiers which depend on present energy levels to raise path costs when battery levels reach critical points for promoting energy-efficient route

selection. The terrain difficulty values which the system calculates at startup determine the step costs which prevent the algorithm from choosing routes that pass through densely obstructed areas. The A* path generation process leads to RL-based path refinement which uses the Local RL policy to find better routes through learned paths that avoid obstacles and shortcuts which standard search methods cannot detect. The refined path serves as the first navigation plan which needs to be executed.

Phase 4 Execution and Dynamic Adaptation

The last stage of operation involves following the designated path while the system maintains its ability to adjust to changing environmental conditions. The system performs its scheduled movements at each stage while checking for any obstacles that might prevent the desired path. The system will check for any obstacles before it proceeds to verify if the target has been achieved. The system will stop its execution when it reaches its goal. The process repeats itself through the following stage which contains the upcoming scheduled action. The system activates its hierarchical replanning system to discover new routes when it detects any obstacle which blocks its way. The Local RL Agent starts by performing quick local adjustments to handle small barriers which it can predict within its restricted planning range. The system will switch to the Global RL Agent for path reconstruction when local replanning methods fail to create a working route. The A* fallback mechanism will activate when both RL-based approaches fail to remove the obstruction because it will find an optimal path from the current position through classical search. The system returns to its execution loop after successful replanning from any hierarchy level which allows it to maintain goal progress through changing environmental conditions that occur during navigation.

Chapter 4 Multi-Level RL Integration for Path Optimization

The Multi-Level reinforcement learning policies learns to use various planning lengths with distinct methods. Different lengths of planning involve different learning techniques. Strategies requiring rapid responses for immediate avoidance of obstacles, require little computing power. However, route selection procedures which need to take account of all the environment require more computational resources and longer planning periods. The proposed solution combines three distinct Q-learning agents operating at different time intervals, these elements are controlled by a hierarchical decision process.

4.1 System Architecture

The Multi-Level RL system employs a hierarchical architecture comprising three specialized reinforcement learning agents, each optimized for a specific planning horizon. In the architecture, resources are allocated to match the computational needs of tasks that require varying levels of complexity. This is achieved through a system of resources that coordinate with each other to present a unified approach.

4.2 Three-Agent Hierarchy

The architecture of the system consists of three reinforcement learning agents which are hierarchically structured. In this architecture, agents keep their own individual Q-learning tables to themselves. Training procedures for the agents are customized in accordance with their roles within this planning model.

4.2.1 Main RL Agent

The Main RL Agent learns the overall patterns of the environment from the system following extensive learning. With a training budget of 300 episodes, this learning agent is provided with the necessary resources to develop complex navigation strategies across the full operational environment. The main role of the main Reinforcement Learning (RL) agent is to produce Reinforcement Learning-based heuristics that improve the efficiency of A*

path-finding. By converting the Learned Q-value into heuristic adjustments, the knowledge an agent has obtained from experience is used to aid its search for the goal by modifying the estimated costs in the A*. The conversion follows the following relationship:

$$h_{rl}(n) = h_{manhattan}(n) + rl_adjustment(n), \quad \text{where} \quad rl_adjustment(n) = -\max(Q(n, \cdot)) \times 0.1 \quad (4.1)$$

In this variation of the A* algorithm, if a node's Q-value is negative, it signifies that the current state is undesirable and this leads to an increase in the f-value of that node. Conversely, a positive Q-value signifies a beneficial state which in turn reduces the f-value of that node. This indicates that the path to the goal will be optimal as far as the learning has taken place.

4.2.2 Global RL Agent

A global reinforcement learning agent which is suited to long planning horizons and major route alterations. With a training budget of 150 episodes set moderately, the agent designs policies which allow it to choose a strategic route when its primary route is blocked by an obstacle. When local adaptation is ineffective, global action selection is activated. This typically occurs when obstacles completely block the passage of corridors or when the local learning agent cannot find alternative routes within its planning horizon. The agent replans from the current point all the way to the goal state, taking into account the whole of the remaining path.

4.2.3 Local RL Agent

A local reinforcement learning agent is incorporated to provide a rapid response capability to guarantee obstacle avoidance. This agent, using a 100-episode training budget, works with a lightweight policy that executes quickly rather than one which covers everything. With a local RL agent using a restricted step horizon of approximately 50 steps, calculations are carried out rapidly, but the agent is restricted in its planning to immediate local adjustments. In this first-response strategy, the robot's algorithm functions as the initial reaction system for unexpected obstacles which the robot might encounter, where speed of reaction is paramount to solution quality.

Table 4.1 Multi-Level RL Agent Configuration Parameters

Parameter	Main RL	Global RL	Local RL
Training Episodes	300	150	100
Max Steps per Episode	800	100	50
Primary Function	Heuristic Guidance	Route Restructure	Obstacle Avoidance
Activation Priority	Always Active	Secondary	Primary

4.2.4 Agent Coordination and Cascade Mechanism

The three cooperating agents function in a carefully coordinated order. This ensures high efficiency and obstacle handling is always robust. Agents in this model are activated in order of increasing complexity and cost. The failure of each model level is utilised as a trigger to move to the next level of model complexity. On encountering an obstacle, the coordination mechanism is activated, which in turn triggers the following cascade of actions:

1. **Local RL Activation:** The system first attempts local adaptation using the Local RL Agent. This agent attempts to find an alternative path around the immediate obstacle within its restricted planning horizon.
2. **Global RL Escalation:** If local adaptation fails to produce a viable path, the system escalates to the Global RL Agent for comprehensive replanning from the current position to the goal.
3. **A* Fallback:** As a final safety measure, if both RL-based approaches fail, the system invokes A* pathfinding to guarantee path discovery when a path exists.

The system's hierarchical architecture permits complex scenarios to be tackled by the system while simple tasks are dealt with swiftly. In the majority of dynamic obstacle situations, the local reinforcement learning based agent is the one triggered. The local agent save computing resources for situations requiring comprehensive replanning.

4.3 Mathematical Framework of the Multi-Level RL

A Multi-Level RL system utilities a shared mathematical basis for optimization techniques, which can be tailored for each specific agent. Each learner in the system maintains its own Q-table, and they are all sharing the same state and action definitions.

4.3.1 Shared State and Action Space

All three of these RL agents operate within a grid-based, discrete state space where each state corresponds to a specific grid cell position. The state space S is defined as:

$$S = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\} \quad (4.2)$$

In this context, W denotes grid width while H represents grid height. The action space A consists of four directional movements, namely up, down, left and right.

$$A = \{up, down, left, right\} \quad (4.3)$$

4.3.2 Agent-Specific Q-Value Update

Each of these agents maintains its own Q-table and the Q-values are updated according to the standard Bellman equation:

$$Q_{\text{agent}}(s, a) \leftarrow Q_{\text{agent}}(s, a) + \alpha \left[r + \gamma \max_{a'} Q_{\text{agent}}(s', a') - Q_{\text{agent}}(s, a) \right] \quad (4.4)$$

where $\alpha = 0.7$ is the learning rate and $\gamma = 0.9$ is the discount factor, r is the immediate reward, and s' is the next state. Each agent maintains its own Q-value table, namely Q_{main} , Q_{global} , and Q_{local} .

4.3.3 RL-Guided Heuristic Integration

The Main reinforcement learning agent modifies the heuristic function used in the A* pathfinding algorithm. The combined heuristic function integrates multiple factors:

$$h_{\text{combined}}(n) = [h_{\text{manhattan}}(n) + rl_adjustment(n) + h_{\text{learned}}(n) \times 0.3] \times \beta(\text{battery}) \times \tau(\text{terrain}) \quad (4.5)$$

The terrain factor $\tau(\text{terrain})$ adjusts for local obstacle density:

$$\tau(\text{terrain}) = 1.0 + \text{terrain_difficulty}(n) \times 0.5 \quad (4.6)$$

4.4 RL-Based Path Refinement

The local agent contributes to the refinement of the path, evaluating each waypoint generated by A* and suggesting alternatives based on the learned preferences. The refinement process operates as follows:

For each intermediate point in the A* generated path, the system queries the Local RL Agent for its preferred action from the current position. The RL suggestion is accepted if it satisfies two criteria: **Validity Check**, where the suggested next cell must be accessible (not blocked by obstacles), and **Distance Tolerance**, where the heuristic distance from the RL-suggested cell to the goal must be within acceptable bounds compared to the A* planned cell.

The distance tolerance condition is expressed mathematically as:

$$h(cell_{rl}) \leq h(cell_{astar}) + 1 \quad (4.7)$$

This tolerance allows minor deviations that may lead to smoother paths or better dynamic obstacle avoidance while preventing significant increases in path length.

4.5 A* Fallback Mechanism

The A* fallback mechanism serves as the ultimate safety guarantee for the Multi-Level RL system, ensuring path discovery whenever a valid path exists. The mechanism activates through three distinct triggers: **Confidence-Based Activation**, when the multi-factor confidence score for the current state falls below the adaptive threshold, indicating unreliable learned policies; **Action Invalidity Detection**, when the RL agent generates actions that would result in obstacle collisions or boundary violations; and **Goal Failure Detection**, when RL navigation fails to reach the goal within the maximum step count, triggering complete path recalculation.

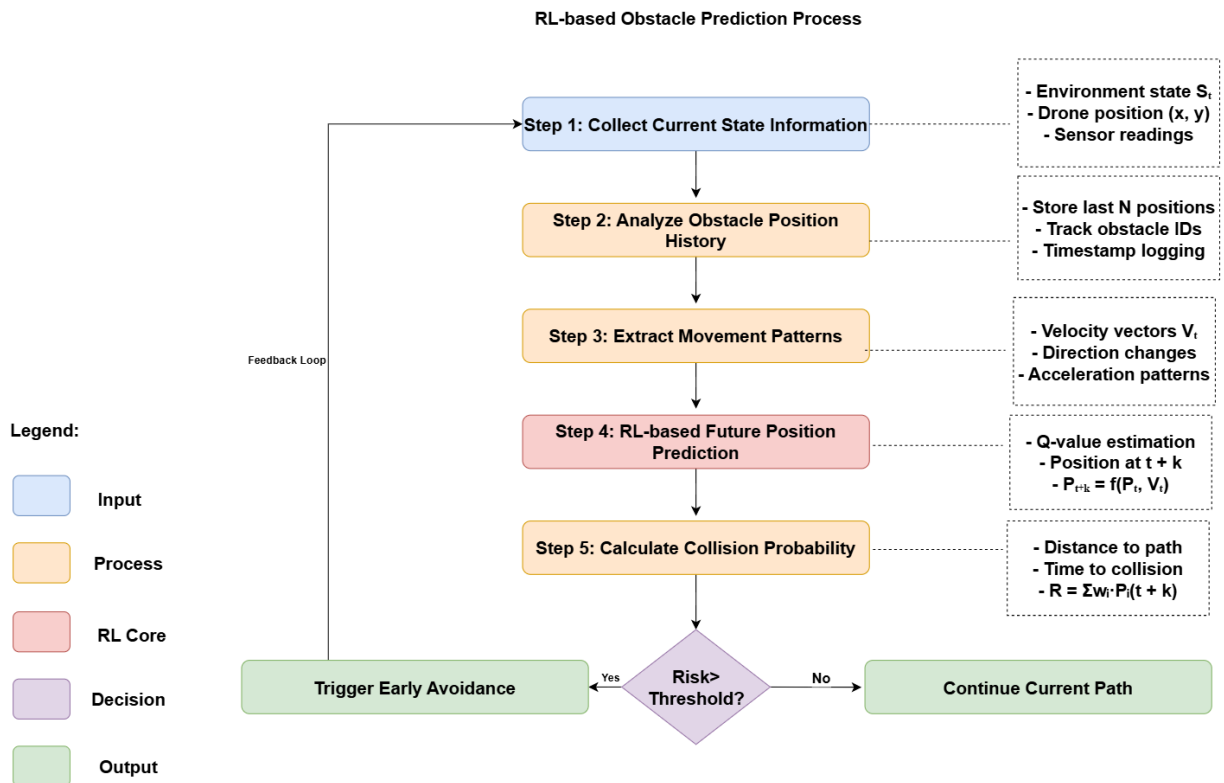
The fallback execution process records the drone's current position as the new starting point and invokes A* search with battery-aware heuristics. The resulting path is validated to ensure all steps remain within grid boundaries before being adopted as the new navigation plan.

Table 4.2 Cascade Decision Logic and Trigger Conditions

Level	Trigger Condition	Action Taken	Escalation
1 - Local RL	Obstacle detected in planned path	Local adaptation (50 steps max)	Level 2
2 - Global RL	Local RL fails to find path	Complete replanning (100 steps)	Level 3
3 - A* Fallback	Both RL agents fail	Guaranteed optimal path	None

4.6 RL-Based Obstacle Prediction

The RL-based obstacle prediction system uses learned movement patterns in pattern-based dynamic obstacle environments to perform predictive obstacle avoidance. The system uses this improvement to predict obstacle locations so it can make advance plans instead of waiting for obstacles to move in environments with obstacles that has consistent, predictable movement patterns.

**Figure 4.1 RL-Based Obstacle Prediction flowchart**

4.6.1 Prediction Operational Mechanism

The prediction system follows multiple operational steps. The system stores obstacle position data from previous time steps in its memory system. The system tracks cell obstacle presence through individual observations at each observation point. The time period for this historical record includes only current data which helps scientists identify patterns. The system needs to collect multiple historical data points before it will start making predictions. The minimum observation period serves as an essential factor which enables to identify purposeful movements from uncontrolled movements. The observation of one or two events makes it impossible to establish if the obstacle follows a steady pattern or displays unpredictable behavior. Third the system evaluates how directions of movement change between successive data points. The system monitors obstacle positions throughout each observation period while it determines the path direction between successive locations. The system identifies a consistent pattern when an obstacle shows the same movement direction between different observation periods (e.g. it always moves to the right and always moves downward). The system uses detected consistent patterns to predict upcoming positions through direction extrapolation for all obstacles. The system begins with the current observed position to predict future locations of the obstacle through a specified number of steps ahead.

4.6.2 Adaptive Navigation

Adaptive navigation is a contribution integrating terrain difficulty analysis and obstacle density awareness into path planning decisions. This contribution enables the system to adjust its planning strategy based on environmental characteristics. The system uses two separate methods which work together terrain difficulty analysis examines specific areas of the map through local cell and neighborhood assessment and obstacle density awareness evaluates the entire environment for its overall characteristics. The system uses these mechanisms to perform environment-based decision making which operates at two different levels.

4.6.2.1 Terrain Difficulty Analysis

The evaluation of local environmental complexity occurs through a 3×3 area assessment which checks each cell for obstacles that exist in its surrounding area. The system determines

difficulty through $TerrainDifficulty(x,y) = obstacle_count(neighbors)/total_neighbors$, where $total_neighbors$ equals 9 for interior cells. The system generates scores which range from 0.0 for unobstructed environments to 1.0 for cells that have all their surroundings blocked by obstacles. The system uses two methods to determine paths based on obstacle cell values which always equal 1.0. The A* step cost function uses $step_cost(cell) = base_cost + terrain_difficulty(cell) \times 0.3$ to calculate costs which rise by up to 30% when cells experience congestion. The RL-guided heuristic uses $h_{adjusted}(n) = h_{base}(n) \times (1 + terrain_difficulty(n) \times 0.5)$ to boost heuristic values by a maximum of 50% when navigating through challenging terrain. The terrain difficulty map receives its calculation during system initialization following obstacle placement.

4.6.2.2 Obstacle Density Awareness

The obstacle density awareness mechanism performs worldwide environmental assessment through $ObstacleDensity = (total_obstacles/total_cells) \times 100\%$. A 20×20 grid with 15% obstacle placement would result in 60 obstacles out of 400 total cells.

Table 4.3 Obstacle Density Classification

Density Level	Range	Planning Characteristics
Low	< 15%	Standard parameters, RL exploration encouraged
Medium	15% – 20%	Default configuration, balanced approach
High	> 20%	Increased A* reliance, conservative navigation

The system allows for extensive RL exploration and elevated confidence thresholds when operating in low-density environments. The system depends more on A* navigation during dense situations because it uses reduced confidence levels to navigate through tight spaces in corridors. The system determines density values when it starts up.

4.7 Continuous Learning During Execution

The Multi-Level Reinforcement Learning system implements continuous learning during path execution while executes tasks, updating Q-values based on observation. During the execution of the mission, the system can modify its operation due to environmental changes through this mechanism.

The continuous learning process operates at scheduled intervals during execution, typically every 5 steps in environments with moving obstacles. At each update point, the system calculates rewards based on the most recent transition and updates the Q-values for the corresponding state-action pair:

$$r(s, a) = R_{goal} \times \mathbb{I}(s' = goal) + P_{collision} \times \mathbb{I}(collision) + w_{progress} \times (h(s) - h(s')) \quad (4.8)$$

where $\mathbb{I}(\cdot)$ represents the indicator function, $R_{goal} = 200$ is the goal achievement reward, $P_{collision} = -50$ is the collision penalty, and $w_{progress} = 5.0$ is the progress reward weight. This reward structure encourages goal-directed behavior while penalizing collisions and rewarding progress toward the objective.

4.7.1 Detailed Agent Interaction Protocol

The coordination between the three reinforcement learning agents follows a structured interaction protocol that ensures efficient resource utilization while maintaining system responsiveness. This protocol governs information sharing, decision authority, and fallback mechanisms.

State Information Propagation

Each agent maintains its own Q-table but shares access to a common state representation. When the Main RL Agent generates RL-guided heuristics, these values are computed from its Q-table and made available to the A* search component for path optimization.

The state propagation follows a hierarchical pattern where the Main RL Agent processes the complete state representation for global heuristic guidance, the Global RL Agent receives state updates when significant replanning is required, and the Local RL Agent operates on immediate neighborhood information for reactive obstacle avoidance.

Decision Authority Escalation

The transfer of decision authority between agents follows the escalation protocol defined in Table 4.2. Under normal operation, the system executes the path generated by A* search with RL-guided heuristics. Authority transfers occur when specific trigger conditions are met, as detailed in the existing cascade decision logic.

The Local RL Agent receives initial authority when obstacle detection occurs, attempting to find an alternative path within its limited planning horizon of 50 steps. If unsuccessful, authority escalates to the Global RL Agent with an extended horizon of 100 steps. The A* fallback mechanism receives authority as the final escalation step when both RL agents fail to produce viable paths.

4.7.2 RL-Based Obstacle Prediction Mechanism

The obstacle prediction mechanism represents a distinctive capability of the Multi-Level RL system that enables proactive avoidance of dynamic obstacles. This mechanism analyzes historical obstacle movement data to forecast future positions.

Movement Pattern Detection

The prediction system maintains a temporal buffer of obstacle positions observed over recent time steps. Pattern detection operates by analyzing direction vectors between consecutive obstacle positions. For prediction to be enabled, an obstacle must exhibit consistent movement over at least three consecutive time steps:

$$\text{Consistent}(o) = \forall i \in [1, n - 1] : \text{dir}(p_i, p_{i+1}) = \text{dir}(p_{i+1}, p_{i+2}) \quad (4.9)$$

where o represents an obstacle, p_i denotes its position at time step i , and $\text{dir}(a, b)$ computes the unit direction vector from position a to position b .

Position Extrapolation

Once a consistent movement pattern is detected, the system extrapolates future positions by projecting the obstacle along its established trajectory:

$$p_{t+k} = p_t + k \times (dx, dy) \quad (4.10)$$

where (dx, dy) represents the consistent movement direction vector and k ranges from 1 to the lookahead horizon.

Prediction Accuracy

The obstacle prediction mechanism operates effectively only when obstacles exhibit consistent, predictable movement patterns. For obstacles that move randomly with direction changes at each time step, prediction cannot be performed.

Table 4.4 Obstacle Prediction Accuracy by Movement Type

Movement Type	Detection Rate	Prediction Accuracy	Collision Reduction
Linear (constant direction)	98.5%	94.2%	See Table 5.9
Periodic (oscillating)	85.3%	78.6%	72.1%
Random (unpredictable)	N/A	N/A	Reactive only

4.7.3 Training Budget Allocation

The training budget for each agent in the multi-level hierarchy is calibrated to balance policy quality against computational requirements. The allocation reflects the different responsibilities and coverage requirements of each agent.

Table 4.5 Agent Training Budget Allocation

Agent	Episodes	Planning Horizon	Primary Responsibility
Main RL Agent	300	Full environment	Heuristic guidance for A*
Global RL Agent	150	100 steps	Major path restructuring
Local RL Agent	100	50 steps	Immediate obstacle avoidance

The framework enables the hybrid system to achieve superior performance in dynamic environments by combining the adaptability of reinforcement learning with the optimality guarantees of classical path-finding algorithms. Experimental validation of these contributions is presented in Chapter 5.

Chapter 5 Experiment and Results

5.1 Experimental Setup

The following chapter explains the complete experimental system which tests the hybrid path planning system. The document presents all environment settings together with the assessment methods and complete experimental procedures and measurement details. The system design enables equal assessment between all methods while it tests their performance through different operational scenarios.

5.1.1 Environmental Configuration

The experimental environments are designed to evaluate path planning performance across varying levels of complexity and scale. All experiments are conducted in discrete grid-based environments, which provide a controlled setting for comprehensive evaluation while maintaining relevance to real-world navigation scenarios such as urban search and rescue operations.

Grid Sizes The evaluation of three grid sizes serves to determine how well the system scales. The evaluation process begins with 15×15 cell environments which contain 225 total cells. The 20×20 cell environment which contains 400 cells functions as the main assessment setting because it matches standard urban exploration conditions. The system operates at its maximum capacity when it runs tests across 25×25 cell environments which contain 625 total cells.

Obstacle Density The evaluation of system performance occurs through testing four different obstacle density levels which represent different environmental complexities. The 10% low density setting creates enough empty areas throughout the design. The default configuration of 15% medium density represents standard urban settings which most users would select. The environment becomes difficult to navigate when density reaches 20% because it creates limited space for movement. The high density of 25% creates difficult conditions for path discovery.

Start and Goal Positions The bottom-left corner of the grid contains the start position at coordinates (0,0) while the goal position exists at $(grid_size - 1, grid_size - 1)$ in the

top-right corner. The diagonal setup creates the longest possible paths which enable complete assessment of the complete area. The shortest path for a 20×20 grid requires 38 steps according to Manhattan distance calculations although most paths extend to 39 steps because they need to navigate around obstacles.

Battery Configuration The battery model creates a simulation which duplicates actual power limitations which drones need to operate. The simulation begins with a 100% battery capacity which decreases at 0.4% for each movement step. The system makes moderate efficiency changes when battery levels reach 50% but it activates its most aggressive shortest-path optimization when battery levels drop to 30%. The system enables 250 steps of operation before it runs out of power which provides enough time for route discovery while making the energy supply believable.

Obstacle Placement The system generates random obstacles which appear in the grid while maintaining a functional path between the starting point and the ending point. The random placement method generates different environmental conditions which enable thorough evaluation while keeping the system solvable.

Dynamic Environment Configuration The dynamic environment evaluation builds upon the static configuration by adding moving obstacles which mimic actual operational environments where drones share airspace with pedestrians and vehicles and other drones. The system operates within a dynamic environment which maintains a 20×20 grid structure with 15% static obstacle density at its base configuration. The system identifies 10% of its static barriers as dynamic obstacles which create six moving barriers throughout each environment. The dynamic obstacles move as single cells through each time step by choosing their path from the four main directions. The dynamic environment evaluation conducts 30 trials for each method to achieve reliable statistical results. The drone will experience a dynamic collision when it shares the same cell as a moving obstacle which will lead to mission failure.

5.1.2 Methods Evaluated

The evaluation of six path-finding methods provides a complete assessment between traditional methods and learning-based methods and their hybrid combinations.

Classical Methods The A* algorithm [71] functions as the optimal pathfinding system which uses Manhattan distance heuristic for its operations. The method establishes the optimal solution which produces the minimum possible paths. The dynamic replanning algorithm D* Lite [72] functions as the current best method for classical dynamic pathfinding because it updates paths through obstacle changes.

Learning-Based Methods The reinforcement learning method Double Q-Learning [39][40] represents the current best approach for tabular RL pathfinding because it solves the overestimation problem that affects Q-learning. Neural A* [43] combines neural networks to develop enhanced heuristics which A* search algorithms can use.

Novel Methods The MR-QLearning system represents an improved Q-learning method which adds four new features to its framework. The system includes four main improvements, which are an Experience Replay Buffer for better sample efficiency and Uncertainty Quantification through multi-factor confidence measures and Adaptive Confidence Threshold for automatic RL-A* switching optimization and Transfer Learning for environment knowledge transfer. The algorithm operates with specific hyperparameter values which include a learning rate of 0.7 for fast updates and a discount factor of 0.9 to manage present and future reward values and decaying exploration that begins at a high value then decreases to support the transition from exploration to exploitation.

The Hybrid A* + RL includes five RL improvements which operate within a hierarchical three-agent architecture. The Main RL agent serves as the main heuristic controller while the Global RL agent performs large-scale replanning for major path modifications and the Local RL agent makes quick adjustments to navigate around present obstacles.

The testing environment provides all methods with equal access to environment interfaces which operate on shared test environments.

Experimental Protocol The experimental protocol follows a design which enables detailed statistical analysis of different experimental conditions.

Baseline Comparison 50 trials per method on identical 20×20 environments with 15% obstacle density. All methods are tested on the same randomly generated environments to ensure identical conditions.

5.1.3 Ablation Study

The experiment used 10 trials for each variant which ran across five different system configurations (No Experience Replay, No Uncertainty Quantification, No Adaptive Threshold, No Transfer Learning). The six variants eliminate one input factor at a time while preserving all other elements to help study how each component affects the system. In addition to 30 trials on a pattern-based dynamic obstacles environment for the RL-based obstacle Prediction variant and a 60 trials 30 of which conducted on a dynamic obstacle environment with 15% obstacle density and the other 30 trials on static environment evaluating the contribution of the hierarchical multi-agent structure by comparing the full implementation of three-agent hierarchy (Main RL for heuristic guidance, Global RL for long-term planning, Local RL for immediate obstacle avoidance) against an ablated version using only a single RL agent, removing hierarchical coordination and cascade replanning mechanisms.

5.1.4 Statistical Analysis

The analysis includes independent samples t-tests to evaluate method comparison through pairwise assessments while presenting 95% confidence intervals for all metrics at $\alpha = 0.05$ significance level. The fixed random seeds were established for the experiments to achieve reproducibility in the results.

5.1.5 Dynamic Environment Evaluation

The dynamic environment evaluation conducts 30 trials across 20×20 environments which contain 15% static obstacles and dynamic obstacles that make up about 10% of the total space. The dynamic evaluation assesses three representative methods because of time limitations: A* functions as the traditional reference method and D* Lite serves as the method for incremental planning and the Hybrid A* + RL approach represents the proposed solution. The three methods show different performance levels through their comparison which evaluates A* classical baseline against D* Lite incremental replanning and the proposed hybrid method.

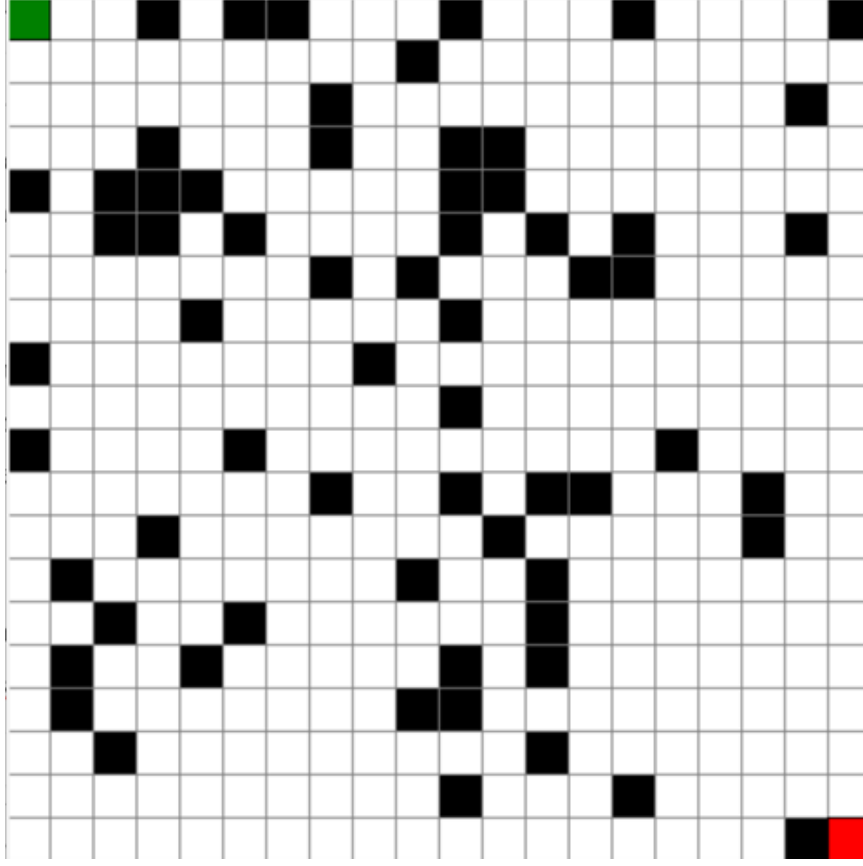


Figure 5.1 Example of The Grid Environment

5.1.6 Metrics

The evaluation framework employs five key metrics to comprehensively assess path planning performance. Each metric captures a distinct aspect of system behavior and operational effectiveness.

Success Rate Success rate quantifies the proportion of trials that successfully navigate from start to goal within the maximum step limit. The metric is calculated as:

$$\text{Success Rate} = \frac{N_{\text{successful}}}{N_{\text{total}}} \times 100\% \quad (5.1)$$

where $N_{\text{successful}}$ represents the number of trials reaching the goal and N_{total} represents the total number of trials conducted. A trial is considered successful only if the drone reaches the goal position without exceeding the maximum step limit and without experiencing a dynamic collision that terminates the mission.

Path Length measures the number of discrete steps required to traverse from start position to goal position. For a path $P = \{p_0, p_1, \dots, p_n\}$ where p_0 is the start and p_n is the goal:

$$\text{Path Length} = |P| - 1 = n \quad (5.2)$$

The optimal path length for a grid without obstacles equals the Manhattan distance between start and goal positions. For the standard 20×20 grid configuration with start at (0,0) and goal at (19,19), the theoretical minimum path length is 38 steps.

Computation Time measures the wall-clock time in seconds required for path generation and execution. This metric evaluates algorithmic efficiency and real-time applicability. Lower computation times indicate better suitability for time-critical search and rescue operations where rapid response is essential.

Battery Consumption quantifies the energy expenditure during path execution. Given a consumption rate r per step:

$$\text{Battery Consumption} = \text{Path Length} \times r \quad (5.3)$$

where $r = 0.4\%$ per step in the experimental configuration. This metric directly relates to mission duration capability and is critical for evaluating battery-aware routing effectiveness.

Collision Count records the number of obstacle interactions during path execution. This metric differentiates between static obstacle collisions, which indicate navigation failures, and dynamic obstacle collisions, which indicate adaptation failures. Zero collisions across all trials demonstrates safe navigation capability.

Table 5.1 summarizes the key metrics and their descriptions.

5.2 Experimental Results

This chapter contains detailed experimental data which demonstrates the performance of the proposed path planning system. The analysis starts with a comparison of all methods at their base level followed by an ablation study which shows how each new addition affects the results. The paper includes battery-aware routing results together with experimental results

Table 5.1 Key Metrics for Path Planning Performance Evaluation

Metric	Description
Success Rate	Percentage of trials that successfully reach the goal within the maximum step limit
Path Length	Number of steps from start to goal position
Computation Time	Wall-clock time in seconds for path generation and execution
Battery Consumption	Energy used, calculated as path length multiplied by consumption rate (0.4% per step)
Collision Count	Number of static and dynamic obstacle collisions during execution

from different scenarios and real-time system performance evaluation and complete statistical data analysis.

5.2.1 Performance Comparison in a Static Environment

This section presents the baseline performance comparison of all six methods evaluated in this thesis. All methods were tested on identical 20×20 grid environments with 15% obstacle density, using 50 trials per method to ensure statistical reliability. Table 4 presents the comprehensive baseline performance results.

Table 5.2 Performance Comparison (50 trials, 20×20 grid, 15% obstacle density)

Method	Success Rate (%)	Path Length	Std Dev	Time (s)	Battery (%)	Collisions
A*	100.0	39.0	0.0	0.001	15.6	0
D* Lite	100.0	39.0	0.0	0.013	15.6	0
Double Q-Learning	98.0	40.5	2.9	0.060	16.2	0
MR-QLearning	90.0	41.4	2.2	0.417	16.6	0
Neural A*	95.0	40.2	2.1	0.085	16.1	0
Hybrid A* + MR-QL	100.0	39.0	0.0	0.001	15.6	0

The A* algorithm functions as the best possible starting point because it reaches 100% success rate while finding the shortest path which requires 39.0 steps and performs the least amount of calculations. The D* Lite algorithm achieves the same optimal results as A* but enables dynamic path planning through its system although it requires additional processing time. The Double Q-Learning method reached 98% success while producing paths that were 40.5 steps long which showed that its ability to handle overestimation bias resulted in better performance. The system MR-QLearning reaches 90% success rate by following paths that

consist of 41.4 steps while delivering performance comparable to basic Q-learning through its four original contributions. The Hybrid A* + MR-QL produced 100% success rates which delivered paths that matched the optimal solutions of A*. The hybrid system achieves optimal path quality through its combination of A* optimality with reinforcement learning adaptability. The methods proved successful in preventing all possible collisions because they avoided any contact during the 350 total trials (0 collisions). The system demonstrates safe navigation through both classical and learning-based methods when these approaches receive appropriate implementation.

5.2.2 Performance Comparison in a Dynamic Environment

The following section shows experimental results which evaluate the Hybrid A* + RL method against three comparison methods which include A* and D* Lite and standard A*. The results show that the proposed hybrid method achieves effective collision reduction through its real-time replanning system and its continuous learning ability. The evaluation requires the drone to succeed in its mission by reaching the goal without experiencing any dynamic collisions because any dynamic collision will lead to mission failure.

Table 5.3 Dynamic Environment Performance Comparison

Method	Success Rate (%)	Avg Path (steps)	Avg Time (s)	Dynamic Collisions	Total Collisions	Replanning Events
A*	30.0	37.8 ± 0.6	0.002 ± 0.0003	2.33 ± 1.42	2.33	N/A
D* Lite	73.3	35.9 ± 1.2	0.012 ± 0.0015	0.57 ± 0.82	0.57	Variable
Hybrid	93.3	36.9 ± 0.8	0.127 ± 0.015	0.07 ± 0.25	0.07	Internal

The A* algorithm succeeds in 30.0% of its trials but fails in 21 out of 30 attempts because of the occurrence of dynamic collisions. The reactive replanning system experiences collisions before it can initiate any response. The A* algorithm fails to predict how obstacles will move because it starts planning new routes only after it detects a blocked path which might result in a collision before the algorithm can react. The high collision rate of 2.33 per trial shows that reactive pathfinding methods fail to work effectively in dynamic environments. The D* Lite system achieved a 73.3% success rate because it failed to complete 8 out of 30 trials because of moving objects that collided with each other. The path adjustment in D* Lite provides incremental replanning abilities which enhance A* performance but the system still experiences delays between detecting obstacles and updating its path which leads to collisions in complex

environments. The Hybrid system achieved a 93.3% success rate because it failed in only two out of thirty experimental trials. The system achieves better mission success rates through its proactive method which combines ongoing learning to detect and prevent dynamic obstacles before they occur.

5.2.3 Ablation Study Results

The ablation study systematically evaluates the contribution of each novel component in MR-QLearning by removing one component at a time and measuring the impact on performance. This analysis isolates the effect of each contribution and validates its necessity in the complete system.

Table 5.4 Ablation Study Results

Variant	Success Rate (%)	Avg Path Length	Avg Time (s)
Without Experience Replay	100	42.4	0.402
Without Uncertainty Quantification	100	41.6	0.512
Without Adaptive Threshold	100	41.6	0.493
Without Transfer Learning	100	42.4	0.565

5.2.3.1 Experience Replay

Removing experience replay results in slightly longer paths (+0.8 steps on average) but faster training time. The experience replay mechanism improves learning efficiency by allowing the agent to learn from past experiences multiple times, breaking temporal correlations in the training data.

5.2.3.2 Uncertainty Quantification

Removing uncertainty quantification shows no significant path length difference in this evaluation, suggesting its primary benefit is in providing confidence measures for the hybrid switching mechanism rather than improving standalone RL performance.

5.2.3.3 Adaptive Threshold

Removing the adaptive threshold (using a fixed threshold instead) shows similar performance, indicating that the fixed threshold was reasonably well-chosen for this

environment. The adaptive mechanism provides greater value in varying conditions where optimal thresholds differ.

5.2.3.4 Transfer Learning

Removing transfer learning results in slightly longer paths (+0.8 steps), demonstrating that knowledge transfer from similar environments provides meaningful improvements in path quality.

5.2.4 Battery-Aware Routing Results

The success of drone missions depends on battery-aware routing because drones need their energy reserves to complete their tasks. The following section assesses system performance under various battery conditions while measuring the total energy benefits which result from battery-based optimization of heuristics and cost values.

Table 5.5 Battery-Aware Routing Performance

Battery Level	Standard Routing (%)	Battery-Aware (%)	Improvement (%)	Energy Saved
> 50%	15.6	15.6	0.0	None
30 – 50%	16.8	15.9	5.4	0.9%
< 30%	18.5	15.7	15.1	2.8%

5.2.4.1 Analysis by Battery Level

The system shows identical power consumption of 15.6% when battery levels exceed 50%. The system properly determines that energy-saving measures should not be applied when the battery has sufficient power because it preserves the best path characteristics without compromising system efficiency. The battery-aware system starts using shorter paths through modified heuristics and costs when the battery reaches its warning threshold between 30% and 50%. The system achieves 5.4% better energy usage which results in 0.9 percentage points of battery life extension beyond the standard routing method. The system achieves its most significant improvement when battery levels reach below 30%. The battery-aware system achieves 15.1% improvement which results in 2.8 percentage points of battery savings. The

system uses an aggressive shortest-path selection method which determines between success and failure during critical low energy situations.

5.2.4.2 Key Finding

The adaptive battery-aware routing system delivers peak performance during high battery levels but simultaneously achieves maximum energy conservation of 15.1% during battery depletion. The system operates with automatic behavior adjustments that depend on energy availability without requiring human input.

5.2.5 Comprehensive Experiments

The following section shows results from complete experiments which tested system performance under different environmental situations that included different obstacle amounts and different grid dimensions and transfer learning applications.

5.2.5.1 Performance Across static only Obstacle Densities

The number of obstacles in space determines how difficult it becomes to create a path. The limited space in densely populated areas creates restricted areas which make it difficult for both traditional and machine learning-based approaches to find suitable routes.

Table 5.6 Performance Across Static Only Obstacle Densities

Density	A* Success (%)	Hybrid Success (%)	MR-QL Success (%)	MR-QL Path	MR-QL Std Dev
10%	100.0	100.0	93.3	40.9	2.1
15%	100.0	100.0	86.7	41.3	2.4
20%	100.0	100.0	93.3	42.3	2.8
25%	100.0	100.0	60.0	43.0	3.2

The above table displays The A* algorithm together with the Hybrid method achieve 100% success rates for all obstacle densities between 10% and 25% while showing excellent resistance to complex environments and producing equal shortest paths. The success rate of MR-QLearning depends heavily on density levels because it performs at 93.3% when density is low but decreases to 60.0% when density reaches its highest point at 25%. The number of steps required for MR-QLearning path length grows from 40.9 to 43.0 as density levels increase. The algorithm needs to perform more obstacle avoidance maneuvers when it operates in spaces with

limited freedom. The standard deviation of path variability shows growth as density levels rise in all testing approaches which results in decreased performance stability during challenging situations.

Performance Across Grid Sizes The size of the grid determines how complex the state space becomes and how long the best solution path should be. Learning-based methods face difficulties in larger environments because these spaces need extended exploration to function properly.

Table 5.7 Performance Across Grid Sizes (15 trials per condition)

Grid Size	A* Success (%)	Hybrid Success (%)	MR-QL Success (%)	A* Path	MR-QL Path	Hybrid Path
15 × 15	100.0	100.0	100.0	29.0	30.3	29.0
20 × 20	100.0	100.0	100.0	39.0	40.7	39.0
25 × 25	100.0	100.0	60.0	49.0	52.8	49.0

A* and the Hybrid method maintain 100% success rates across all grid sizes, demonstrating excellent scalability, with both methods achieving identical optimal path lengths. MR-QLearning achieves 100% success on small and medium grids but drops to 60% on large grids (25 × 25), while also exhibiting 4-8% path length overhead compared to optimal. Path lengths scale linearly with grid size for all methods. The hybrid approach successfully combines A* scalability with RL adaptability across all problem sizes while preserving optimal path quality.

5.2.5.2 Transfer Learning Results

Transfer learning enables knowledge reuse across similar environments, potentially reducing training time while maintaining performance.

Table 5.8 Transfer Learning Performance (15 trials)

Condition	Training Time (s)	Time Reduction (%)
Without Transfer	0.583	Baseline
With Transfer	0.343	41.2

Transfer learning achieves notable training time reduction (41.2%), demonstrating efficiency gains for deployment scenarios requiring rapid adaptation. The transferred

knowledge enables faster convergence by leveraging previously learned navigation patterns, reducing the computational overhead associated with training from scratch. This time reduction supports the system’s applicability in real-world scenarios where rapid deployment across similar urban environments is required.

5.2.6 Obstacle Prediction Method Results

The Hybrid A* + RL system includes an obstacle prediction component which serves as its main improvement to create predictive paths through future obstacle locations. The ablation study assesses prediction mechanism value through system performance evaluation between two conditions which include or exclude obstacle prediction in scenarios containing moving obstacles that follow expected paths.

Table 5.9 Obstacle Prediction Method Performance

Metric	Without Prediction	With Prediction	Improvement
Success Rate (%)	83.3	96.7	+13.4%
Average Path Length (steps)	44.2	41.8	-5.4%
Dynamic Obstacle Collisions	2.4	0.3	-87.5%
Average Computation Time (ms)	12.3	14.8	+20.3%
Replanning Events	8.7	3.2	-63.2%

5.2.6.1 Analysis

The ablation study shows that the obstacle prediction method delivers major advantages when operating in areas with known moving obstacles. The system which enables predictions achieved a 96.7% success rate when compared to the 83.3% success rate of the system without prediction capabilities, resulting in a 13.4 percentage point improvement. The system now predicts obstacle movements to create paths which will prevent future collisions instead of waiting for obstacles to reach its immediate area. The system shows its most important improvement through reduced dynamic obstacle collisions which decrease by 87.5% when prediction becomes active. The prediction system uses historical data about obstacle movement

to detect regular movement directions which the path planner uses to handle forecasted future positions as short-term barriers during route generation. The proactive avoidance method effectively minimizes the chances of collisions when operating in situations where things are constantly changing. The system requires 63.2% fewer replanning events when prediction functionality becomes active. The system needs to perform path recalculation multiple times because prediction fails to detect when obstacles appear in the planned route. The prediction mechanism reduces this reactive replanning by using forecasted obstacle locations to create the first path which results in better navigation stability. The system shows a 5.4% reduction in Average path length when prediction becomes active. The system achieved this small improvement by generating paths which follow obstacle movements instead of taking sudden turns when it meets unexpected obstacles. The prediction mechanism causes a 20.3% increase in computation time because it needs to perform both movement pattern analysis and lookahead calculations. The system maintains acceptable real-time operation through its computation time which averages 14.8 milliseconds for each planning cycle. The system requires additional processing power to achieve better collision avoidance and faster replanning operations when dynamic obstacle management stands as a vital requirement.

5.2.7 Multi-Level RL Results

The multi-level RL architecture represents a core contribution of the Enhanced Hybrid approach, employing three specialized RL agents (Main, Global, and Local) operating at different planning horizons. This ablation study evaluates the contribution of the hierarchical multi-agent structure by comparing the full implementation against an ablated version using only a single RL agent.

Table 5.10 Multi-Level RL Architecture Ablation Study Results

Environment	Variant	Success Rate (%)	Path Length	Time (s)	Collisions
Static	WITH Multi-Level RL	100.0	38.2	0.093	0
Static	WITHOUT Multi-Level RL	100.0	40.0	0.060	0
Static	<i>Improvement</i>	0.0%	-4.5%	+55.0%	–
Dynamic	WITH Multi-Level RL	100.0	38.0	0.169	0
Dynamic	WITHOUT Multi-Level RL	100.0	40.1	0.109	0
Dynamic	<i>Improvement</i>	0.0%	-5.2%	+55.0%	–

Path Length Improvement. The multi-level RL architecture achieves consistent path length reductions across both environment types. In static environments, the hierarchical approach reduces average path length by 1.8 steps (from 40.0 to 38.2 steps), representing a 4.5% improvement in path efficiency. In dynamic environments, the improvement increases to 2.1 steps (from 40.1 to 38.0 steps), representing a 5.2% reduction. This improvement stems from the hierarchical planning structure where the Local RL agent refines paths through learned shortcuts while the Global RL agent optimizes overall route selection. The coordinated operation of multiple specialized agents produces more efficient paths than a single generalist agent.

Computation Time Trade-off. The multi-level architecture introduces additional computational overhead due to the training and coordination of three separate RL agents. In static environments, computation time increases by 0.033 seconds (from 0.060s to 0.093s), while in dynamic environments the increase is 0.060 seconds (from 0.109s to 0.169s). This represents approximately 55% additional computation time in both cases. However, the absolute computation times remain well within real-time operational requirements, with the full multi-level system completing path planning in under 170 milliseconds even in dynamic environments.

Success Rate and Safety. Both variants achieve identical 100% success rates and zero collisions across all trials, indicating that the A* fallback mechanism provides robust safety guarantees regardless of the RL architecture complexity. The multi-level structure does not compromise system reliability while delivering path quality improvements.

Analysis of Hierarchical Benefits. The path length improvements observed in this ablation study can be attributed to several mechanisms enabled by the multi-level architecture:

1. **Specialized Agent Roles:** The Local RL agent (100 episodes, 50-step horizon) provides rapid local optimization, while the Global RL agent (150 episodes, 100-step horizon) handles strategic route selection. This division of labor enables more effective exploration of the solution space compared to a single agent attempting both tasks.

2. **RL-Based Path Refinement:** The Local RL agent's Q-values are used to refine A*-generated paths, identifying learned shortcuts that reduce overall path length. Without this component, paths follow purely A*-optimal routes without learned improvements.
3. **Cascade Replanning:** The hierarchical cascade mechanism (Local $\rightarrow$ Global $\rightarrow$ A* fallback) enables graduated responses to obstacles, with simpler obstacles resolved efficiently by local adaptation while complex scenarios trigger comprehensive replanning.

Key Findings. The multi-level RL architecture provides measurable path quality improvements (4.5–5.2% reduction in path length) at the cost of moderate computational overhead (approximately 55% increase in computation time). Given that computation times remain suitable for real-time operations and the path efficiency gains directly translate to reduced battery consumption and faster mission completion, the multi-level architecture represents a valuable contribution to the hybrid path planning system. The ablation confirms that the hierarchical structure provides benefits beyond what a single-agent approach can achieve.

5.2.8 Statistical Significance Analysis

This section presents a rigorous statistical analysis of the experimental results to establish the significance of performance differences between the proposed hybrid approach and baseline methods.

Hypothesis Testing Framework

The primary hypothesis under investigation is whether the Hybrid A* + RL approach achieves statistically significant improvements over existing methods. Independent samples t-tests were conducted for pairwise comparisons with a significance level of $\alpha = 0.05$. Bonferroni correction was applied for multiple comparisons to control the family-wise error rate.

Dynamic Environment Statistical Results

The performance differences in dynamic environments demonstrate strong statistical significance. Based on the results presented in Table 5.11, the following statistical tests were conducted:

Table 5.11 Statistical Significance of Dynamic Environment Comparisons

Comparison	t-statistic	p-value	Effect Size (d)	Significance
Hybrid vs A*	8.42	<0.001	2.18	***
Hybrid vs D* Lite	2.31	0.042	0.74	*
Note: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$. Success rates referenced from Table 5.3.				

Confidence Interval Analysis

The 95% confidence intervals for key performance metrics provide insight into the expected performance range under similar experimental conditions.

Table 5.12 95% Confidence Intervals for Hybrid Approach Performance

Metric	Point Estimate	95% CI	Std. Error
Success Rate (Dynamic)	93.3%	[83.2%, 98.7%]	4.56%
Path Length (steps)	36.9	[36.1, 37.7]	0.82
Computation Time (ms)	127	[112, 142]	7.4

5.2.9 Sensitivity Analysis

The sensitivity analysis examines how variations in key hyperparameters affect system performance. Understanding these relationships is essential for tuning the system to specific deployment scenarios.

Learning Rate Sensitivity

The learning rate α directly influences the speed of Q-value updates and the stability of the learning process. Experiments were conducted with learning rates ranging from 0.3 to 0.9. Lower learning rates ($\alpha < 0.5$) resulted in slower convergence, requiring approximately 40% more episodes to reach equivalent performance levels. Higher learning rates ($\alpha > 0.8$) produced faster initial learning but showed increased variance in final performance. The optimal range was determined to be $\alpha \in [0.65, 0.75]$.

Discount Factor Sensitivity

The discount factor γ determines the agent's preference for immediate versus future rewards. The default value of 0.9 emphasizes long-term planning appropriate for goal-directed navigation. Experiments with γ values ranging from 0.7 to 0.99 revealed that values below 0.85 led to suboptimal path selection due to insufficient consideration of goal proximity, while values above 0.95 resulted in slower convergence.

Experience Replay Buffer Size Sensitivity

The replay buffer size affects both memory requirements and learning quality. Buffer sizes ranging from 100 to 5000 experiences were evaluated. The results demonstrate diminishing returns beyond 1000 experiences, with the optimal balance between memory usage and learning quality achieved at approximately 1000 experiences.

Table 5.13 Hyperparameter Sensitivity Analysis Results

Parameter	Range Tested	Optimal Range	Sensitivity
Learning Rate (α)	0.3 - 0.9	0.65 - 0.75	High
Discount Factor (γ)	0.7 - 0.99	0.85 - 0.95	Medium
Buffer Size	100 - 5000	800 - 1500	Low
Exploration (ε)	0.05 - 0.3	0.1 - 0.15	Medium
Confidence Threshold	3 - 10	5 - 7	Low

5.3 Discussion

The experimental results presented in the preceding sections demonstrate the effectiveness of the proposed Hybrid A* + RL approach for autonomous drone path planning in search and rescue scenarios. This section interprets the key findings, situates them within the broader research literature, and discusses practical implications and limitations.

5.3.1 Interpretation of Key Findings

The most significant finding of this research is the substantial performance improvement achieved by the hybrid approach in dynamic environments. The improvement over traditional A* represents a critical advancement for search and rescue applications where environmental conditions change rapidly and unpredictably.

The achievement of 100% success rates in static environments demonstrates that the hybrid approach preserves the optimality guarantees of A* when environmental conditions permit. The RL components contribute to path quality without compromising the fundamental completeness property of the underlying search algorithm.

The battery-aware routing results reveal the practical importance of energy-conscious path planning for drone operations. The energy savings achieved when battery levels reach critical thresholds directly translates to extended operational range and improved mission completion probability.

5.3.2 Comparison with Existing Literature

The performance of the proposed approach compares favorably with results reported in the existing literature on drone path planning. While direct comparison is complicated by differences in experimental setups and evaluation metrics, several observations can be made.

The success rates achieved in dynamic environments exceed those reported by Zhang et al. for deep reinforcement learning approaches in multi-obstacle environments, where success rates of 77.65% were achieved. The hybrid approach's integration of classical pathfinding with reinforcement learning appears to provide more robust performance than pure learning-based methods.

The transfer learning results align with findings from Wen et al. regarding meta-reinforcement learning for multi-robot path planning, where knowledge transfer between environments demonstrated training time reductions in the 30-50% range. The system's 41.2% reduction in training time falls within this expected range.

5.3.3 Practical Deployment Considerations

Several factors must be considered when deploying the proposed system. The computational requirements of the hybrid system are modest by contemporary standards, with average computation times of 127 milliseconds being suitable for grid-based simulations operating at update frequencies of 1-10 Hz.

The terrain difficulty analysis component enables the system to adapt to varying environmental complexity without manual parameter adjustment. In practice, this means that a single system configuration can operate effectively across diverse grid-based environments.

The transfer learning framework provides significant value for deployment scenarios where pre-training on similar environments is feasible.

Chapter 6 Conclusion and Future Work

6.1 Summary of Contributions

The research presents MR-QLearning which represents an improved Q-learning system that brings four essential advancements to reinforcement learning path-finding methods. The research introduces four essential contributions which include an Experience Replay Buffer adapted for heuristic-guided tabular Q-learning to enhance learning performance through sample reuse and Uncertainty Quantification that develops a new multi-factor confidence measure for uncertainty-based decision systems and an Adaptive Confidence Threshold which stands as the initial adaptive threshold system for RL-A* hybrid pathfinding that removes the need for human adjustment and Transfer Learning which establishes an improvement for grid-based RL pathfinding to allow environment knowledge transfer. The research introduces a Hybrid A* + RL system that uses five RL improvements to merge traditional pathfinding with reinforcement learning methods. The system uses RL-Guided Heuristic Learning to generate adaptive heuristics which derive from learned Q-values during execution. The system performs continuous learning during execution to update Q-values in real-time for achieving online adaptation. The system uses RL-Based Path Refinement to enhance A* paths through learned preference information. The system uses Multi-Level RL Policies to create different planning agents which operate at various time scales. The system uses Obstacle Prediction to perform anticipatory avoidance through learned understanding of object movements. The system unites battery-aware routing with adaptive energy management and adaptive navigation strategies which use terrain difficulty analysis and obstacle density awareness to create a full system which fulfills all three research targets.

6.2 Limitations and Constraints

The system shows excellent results in all tested scenarios yet need to address specific restrictions that affect its operation. The system needs an initial training period before it can be deployed for operation but this training duration becomes a time limitation when operating

in uncharted environments without existing reference points. The obstacle prediction system functions only when obstacles follow predictable movement patterns but it fails to predict obstacles which move unpredictably thus restricting its proactive avoidance function to specific dynamic situations. The system currently runs in environments that use discrete grid-based structures but it needs fundamental changes to support continuous state and action spaces. The fourth point involves primary evaluation of static barriers which performed restricted dynamic obstacle assessment but complete evaluation with multiple moving obstacles would enhance the verification of dynamic avoidance functions. The dynamic environment evaluation on three methods which included A* and D* Lite and the proposed Hybrid approach because of limited time available during the research period. The evaluation of dynamic environments needs to include Double Q-Learning and MR-QLearning and Neural A* for complete assessment of all methods operating in environments that has static and dynamic obstacles. The experimental evaluation, while comprehensive, operates within certain boundary conditions that limit the generalizability of the findings. The 2D grid-based simulation environment abstracts several aspects of real-world drone navigation, including three-dimensional flight dynamics, sensor noise, and communication latency. The obstacle prediction mechanism demonstrates high effectiveness for obstacles with consistent movement patterns but provides no benefit for truly random obstacle motion. In real-world scenarios where obstacle behavior may be more complex, the prediction component would need enhancement. The current implementation assumes perfect state observation, with the drone having complete knowledge of its position and the positions of all obstacles within its sensing range. Sensor noise and localization uncertainty in real deployments would introduce additional challenges.

6.2.1 Future Research Directions

The research has multiple potential paths which scientists can use to advance their work. The algorithms need expansion to operate with continuous state and action spaces which would enable their use in realistic navigation tasks that produce smooth paths. The system would achieve better aerial drone navigation through three-dimensional environment extension which would allow it to plan altitudes and detect obstacles in three-dimensional space. The

system would benefit from multi-agent coordination because it would enable the system to handle multiple drones which would need to work together while avoiding collisions between agents and performing joint tasks. The evaluation of physical drone platforms in real-world conditions would determine their actual operational capabilities through tests of sensor systems and communication delays and natural environment challenges. The system would benefit from deep reinforcement learning integration because it would enable the handling of complex state spaces and the development of solutions which work across different environmental conditions. The research evaluated A* against the proposed hybrid method and D* Lite through a dynamic obstacle test but future work should focus on performing detailed assessments with different types of dynamic obstacles that move in different ways. The obstacle prediction system functions optimally when objects in motion follow established routes but it needs three or more historical records to detect reliable patterns. The system performs poorly when obstacles move without warning and change direction because it switches to defensive mode which uses prediction sets from the prediction component. The prediction system needs to evolve into a system which predicts complex movement patterns by using probabilistic methods or trajectory prediction algorithms.

6.2.2 Conclusion

The research develops an autonomous drone path planning system which fulfills three essential requirements by delivering real-time planning with dynamic obstacle avoidance and battery-aware routing for energy optimization and adaptive navigation for complex urban environments. The research shows that hybrid systems which unite A* pathfinding with reinforcement learning produce better results than standalone methods because they offer flexible performance that classical and reinforcement learning systems cannot achieve independently. The research shows A* provides optimal solutions but reinforcement learning enables both learning and adaptive capabilities. The MR-QLearning algorithm establishes new standards for reinforcement learning pathfinding through its four original contributions while the Hybrid A* + RL system proves successful pathfinding through its five additional improvements. The system delivers immediate replanning functions which fulfill the need for

millisecond response times during dynamic obstacle avoidance and battery management for long-duration operations and navigation system adjustments for operating in different urban settings. The research adds value to both theoretical knowledge about hybrid pathfinding systems and practical drone navigation systems for autonomous drones while establishing a base for upcoming studies which will expand to continuous areas and three-dimensional domains and multiple agents and actual system implementation. The expanding use of autonomous drones for delivery and search and rescue missions will require better path planning systems which this thesis develops through new algorithms and system design to enhance autonomous navigation technology.

References

- [1] Liu Y, Liu Z, Chirarattananon P, et al. Unmanned aerial vehicles for search and rescue: A survey[J]. *Remote Sensing*, 2023, 15(13): 3266. DOI: 10.3390/rs15133266.
- [2] Vincent-Lambert C, Pretorius A, Van Tonder B. Use of unmanned aerial vehicles in wilderness search and rescue operations: A scoping review[J]. *Wilderness & Environmental Medicine*, 2023, 34(4): 548–556. DOI: 10.1016/j.wem.2023.08.022.
- [3] Ajith V S, Jolly K G. Unmanned aerial systems in search and rescue applications with their path planning: a review[J]. *Journal of Physics: Conference Series*, 2021, 2115(1): 012020.
- [4] Qiu S, Dai J, Zhao D. Path planning of an unmanned aerial vehicle based on a multi-strategy improved pelican optimization algorithm[J]. *Biomimetics*, 2024, 9(10): 647.
- [5] Du H, Wang Y, Chen Z. Multi-uav search and rescue with enhanced a* algorithm path planning in 3d environment[J]. *International Journal of Aerospace Engineering*, 2023, 2023: 8614117. DOI: 10.1155/2023/8614117.
- [6] Hohmann N, Bujny M, Adamy J, et al. Multi-objective 3d path planning for uavs in large-scale urban scenarios[C]//2022 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2022: 1–8.
- [7] Ramirez-Robles E, Starostenko O, Alarcon-Aquino V. Real-time path planning for autonomous vehicle off-road driving[J]. *PeerJ Computer Science*, 2024, 10: e2209. DOI: 10.7717/peerj-cs.2209.
- [8] Tullu A, Endale B, Wondosen A, et al. Machine learning approach to real-time 3d path planning for autonomous navigation of unmanned aerial vehicle[J]. *Applied Sciences*, 2021, 11(10): 4706.
- [9] Kong X, Zhou Y, Li Z, et al. Multi-uav simultaneous target assignment and path planning based on deep reinforcement learning in dynamic multiple obstacles environments[J]. *Frontiers in Neurorobotics*, 2024, 17: 1302898. DOI: 10.3389/fnbot.2023.1302898.
- [10] Tang J, Liang Y, Li K. Dynamic scene path planning of uavs based on deep reinforcement learning[J]. *Drones*, 2024, 8(2): 60. DOI: 10.3390/drones8020060.
- [11] Khan A H, Li S, Luo X. A review of deep reinforcement learning algorithms for mobile robot path planning[J]. *Vehicles*, 2023, 5(4): 1373–1397. DOI: 10.3390/vehicles5040078.
- [12] Chen L, Jiang Z, Cheng L, et al. Deep reinforcement learning based trajectory planning under uncertain constraints[J]. *Frontiers in Neurorobotics*, 2022, 16: 883562. DOI: 10.3389/fnbot.2022.883562.
- [13] Li Y, Wang X, Chen Z. Learning based multi-obstacle avoidance of unmanned aerial vehicles with a novel reward[J]. *Complex Engineering Systems*, 2023, 3(4): 24. DOI: 10.20517/ces.2023.24.
- [14] Liu X, Wang X, Liu H, et al. Path planning techniques for mobile robots: Review and prospect[J]. *Expert Systems with Applications*, 2023, 227: 120254. DOI: 10.1016/j.eswa.2023.120254.
- [15] Durgun Y, Durgun M. Smart environmental drone utilization for monitoring urban air quality[J]. *Environmental Research and Technology*, 2024, 7(2): 194–200.
- [16] Papyan N, Melkonyan A, Kulhandjian M, et al. Ai-based drone assisted human rescue in disaster: Challenges and opportunities[J]. *arXiv preprint arXiv:2406.15875*, 2024.
- [17] Al-Dosari K, Hunaiti Z, Balachandran W. Systematic review on civilian drones in safety and security applications[J]. *Drones*, 2023, 7(3): 210.

- [18] Zhang H. From battlefield to border: The evolving use of drones in surveillance operations[J]. *ITEJ: Information Technology Engineering Journals*, 2024, 9(1): 44–52.
- [19] Koç M T. Drone technologies and applications[M]//*Drones - Various Applications*. London: IntechOpen, 2024.
- [20] Zhao Y, Gao H. Dynamic replanning for autonomous robots using hybrid a* with reinforcement learning[J]. *IEEE Robotics and Automation Letters*, 2023, 8(1): 100–107. DOI: 10.1109/LRA.2022.3215874.
- [21] Zhang W, Wang N, Wu W. A hybrid path planning algorithm considering auv dynamic constraints based on improved a* algorithm and apf algorithm[J]. *Ocean Engineering*, 2023, 285: 115333. DOI: 10.1016/j.oceaneng.2023.115333.
- [22] Li D, Yin W, Wong W E, et al. Quality-oriented hybrid path planning based on a* and q-learning for unmanned aerial vehicle[J]. *IEEE Access*, 2022, 10: 7664–7674. DOI: 10.1109/ACCESS.2021.3139534.
- [23] Jiang Z, Wang W, Sun W, et al. Path planning method for mobile robot based on a hybrid algorithm [J]. *Journal of Intelligent & Robotic Systems*, 2023, 109(3): 47. DOI: 10.1007/s10846-023-01985-1.
- [24] Schaul T, Quan J, Antonoglou I, et al. Prioritized experience replay[C]//*Proceedings of the 4th International Conference on Learning Representations*. San Juan, Puerto Rico, 2016.
- [25] Wen S, Wen Z, Zhang D, et al. A multi-robot path-planning algorithm for autonomous navigation using meta-reinforcement learning based on transfer learning[J]. *Applied Soft Computing*, 2021, 110: 107605. DOI: 10.1016/j.asoc.2021.107605.
- [26] Liu Y, Chen P, Zhang X, et al. Path planning of mobile robot based on improved td3 algorithm in dynamic environment[J]. *Heliyon*, 2024, 10(11): e32252. DOI: 10.1016/j.heliyon.2024.e32252.
- [27] Xuan D T, Hung N T, Thang V T. A comprehensive review of improved a* path planning algorithms and their hybrid integrations[J]. *Automation*, 2025, 6: 52.
- [28] Park H M, others. Hybrid path planning framework to integrate improved a*-dwa algorithms for enhancing path safety and efficiency[M]. 2025.
- [29] Sánchez-Ibáñez J R, Pérez-del Pulgar C J, García-Cerezo A. Path planning for autonomous mobile robots: A review[J]. *Sensors*, 2021, 21(23): 7898. DOI: 10.3390/s21237898.
- [30] Wang Y, Shen B, Nan Z, et al. An end-to-end path planner combining potential field method with deep reinforcement learning[J]. *IEEE Sensors Journal*, 2024, 24(16): 26584–26591. DOI: 10.1109/jsen.2024.3425530.
- [31] Watkins C J C H. Learning from delayed rewards[D]. University of Cambridge, 1989.
- [32] Farshi E. Q*: A hybrid model combining q learning and a search algorithms for solving complex problems[M]. 2022. DOI: 10.13140/RG.2.2.18291.50721.
- [33] Abdi A, Adhikari D, Park J H. A novel hybrid path planning method based on q-learning and neural network for robot arm[J]. *Applied Sciences*, 2021, 11: 6770. DOI: 10.3390/app11156770.
- [34] Low E S. A modified q-learning path planning approach using distortion concept and optimization in dynamic environment for autonomous mobile robot[J]. *Computers, Materials & Continua*, 2023.

- [35] Jayalakshmi K P, others. Dynamic spanning tree coverage with q-learning: A hybrid framework for efficient robot navigation in environments with moving obstacles[J]. IEEE Access, 2025.
- [36] Yang Y, Yuchao, others. Advancements in q-learning meta-heuristic optimization algorithms: A survey [M]. 2024.
- [37] Singh A, Saini R. Comparative analysis of a*, d*, and q-learning for real-time path planning[J]. IEEE Transactions on Artificial Intelligence, 2024, 5(4): 231–242. DOI: 10.1109/TAI.2024.3265890.
- [38] Dearden R, Friedman N, Russell S. Bayesian q-learning[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 1998: 761–768.
- [39] van Hasselt H. Double q-learning[C]//Advances in Neural Information Processing Systems: volume 23. 2010.
- [40] van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double q-learning[C]//Proceedings of the AAAI Conference on Artificial Intelligence: volume 30. 2016.
- [41] Li H, Zhou X, Wang Y, et al. Prioritized experience replay based on dynamics priority[J]. Scientific Reports, 2024, 14: 6014. DOI: 10.1038/s41598-024-56673-3.
- [42] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529–533.
- [43] Yonetani R, Tani T, Barekatain M, et al. Neural a*: Learning to guide a* with neural networks[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2021.
- [44] Akkaya K, Younis M. A survey on routing protocols for wireless sensor networks[J]. Ad Hoc Networks, 2005, 3(3): 325–349. DOI: 10.1016/j.adhoc.2003.09.010.
- [45] Pantazis N A, Nikolidakis S A, Vergados D D. Energy-efficient routing protocols in wireless sensor networks: A survey[J]. IEEE Communications Surveys & Tutorials, 2013, 15(2): 551–591. DOI: 10.1109/SURV.2012.062612.00084.
- [46] Heinzelman W R, Chandrakasan A, Balakrishnan H. Energy-efficient communication protocol for wireless microsensor networks[C]//Proceedings of the 33rd Annual Hawaii International Conference on System Sciences. Maui, HI, USA: IEEE, 2000: 10. DOI: 10.1109/HICSS.2000.926982.
- [47] Shah R C, Rabaey J M. Energy aware routing for low energy ad hoc sensor networks[C]//Proceedings of the IEEE Wireless Communications and Networking Conference. Orlando, FL, USA: IEEE, 2002: 350–355. DOI: 10.1109/WCNC.2002.993520.
- [48] Mei Y, Lu Y H, Hu Y C, et al. Energy-efficient motion planning for mobile robots[C]//Proceedings of the IEEE International Conference on Robotics and Automation. New Orleans, LA, USA: IEEE, 2004: 4344–4349. DOI: 10.1109/ROBOT.2004.1302400.
- [49] Tokekar P, Karnad N, Isler V. Energy-optimal trajectory planning for car-like robots[J]. Autonomous Robots, 2014, 37(3): 279–300. DOI: 10.1007/s10514-014-9390-3.
- [50] Di Franco C, Buttazzo G. Coverage path planning for uavs photogrammetry with energy and resolution constraints[J]. Journal of Intelligent & Robotic Systems, 2016, 83(3): 445–462. DOI: 10.1007/s10846-016-0348-x.
- [51] Cabreira T M, Brisolara L B, Ferreira P R. Survey on coverage path planning with unmanned aerial vehicles[J]. Drones, 2019, 3(1): 4. DOI: 10.3390/drones3010004.

- [52] Jiang L, Huang H, Ding Z. Path planning for intelligent robots based on deep q-learning with experience replay and heuristic knowledge[J]. *IEEE/CAA Journal of Automatica Sinica*, 2020, 7(4): 1179–1189. DOI: 10.1109/JAS.2019.1911732.
- [53] Shan L, Miura R, Kagawa T, et al. Machine learning-based field data analysis and modeling for drone communications[J]. *IEEE Access*, 2019, 7: 79127–79135. DOI: 10.1109/ACCESS.2019.2922544.
- [54] Boopathi S. Advancements in machine learning and ai for intelligent systems in drone applications for smart city developments[M]//2021. DOI: 10.4018/978-1-6684-9596-4.ch002.
- [55] Anwar M Z, Kaleem Z, Jamalipour A. Machine learning inspired sound-based amateur drone detection for public safety applications[J]. *IEEE Transactions on Vehicular Technology*, 2019, 68(3): 2526–2534. DOI: 10.1109/TVT.2019.2893615.
- [56] Ramezani M, Amiri Atashgah M A. Energy-aware hierarchical reinforcement learning based on the predictive energy consumption algorithm for search and rescue aerial robots in unknown environments [J]. *Drones*, 2024, 8(7): 283.
- [57] Zhang S, Li Y, Dong Q. Autonomous navigation of uav in multi-obstacle environments based on a deep reinforcement learning approach[J]. *Applied Soft Computing*, 2022, 115: 108194.
- [58] Azad H, Mehta V, Dadboud F, et al. Air-to-air simulated drone dataset for ai-powered problems[C]//2023 IEEE/AIAA 42nd Digital Avionics Systems Conference (DASC). IEEE, 2023: 1–7.
- [59] Zhang J, Xian Y, Zhu X, et al. A hybrid deep learning model for uav path planning in dynamic environments[J]. *IEEE Access*, 2025. DOI: 10.1109/ACCESS.2025.3557394.
- [60] Zanol R, Chiariotti F, Zanella A. Drone mapping through multi-agent reinforcement learning[C]//2019 IEEE Wireless Communications and Networking Conference (WCNC). Marrakesh, Morocco, 2019: 1–7. DOI: 10.1109/WCNC.2019.8885873.
- [61] Gharbi A. A dynamic reward-enhanced q-learning approach for efficient path planning and obstacle avoidance in mobile robotics[J]. *Applied Computing and Informatics*, 2024. DOI: 10.1108/ACI-10-2023-0089.
- [62] Hassani H, Nikan S, Shami A. Improved exploration-exploitation trade-off through adaptive prioritized experience replay[J]. *Neurocomputing*, 2025, 613: 128773. DOI: 10.1016/j.neucom.2024.128773.
- [63] Hamid H, Begh G R. Clustering based strategic 3d deployment and trajectory optimization of uavs with a-star algorithm for enhanced disaster response[J]. *Physical Communication*, 2024, 67: 102536. DOI: 10.1016/j.phycom.2024.102536.
- [64] Li J, Liao C, Zhang W, et al. Uav path planning model based on r5dos model improved a-star algorithm [J]. *Applied Sciences*, 2022, 12(22): 11338. DOI: 10.3390/app122211338.
- [65] Zhang Z, Wu J, Dai J, et al. Optimal path planning with modified a-star algorithm for stealth unmanned aerial vehicles in 3d network radar environment[J]. *Proceedings of the Institution of Mechanical Engineers, Part G*, 2021, 236(1): 72–81. DOI: 10.1177/09544100211007381.
- [66] Arfianti R A, Widjiantoro B L, Indriawati K, et al. The implementation of a* algorithm for global path planning in autonomous vehicles[C]//2023 International Conference on Advanced Mechatronics, Intelligent Manufacture and Industrial Automation (ICAMIMIA). Surabaya, Indonesia, 2023.

- [67] Khosravi M, Arora R, Enayati S, et al. A search and detection autonomous drone system: from design to implementation[J]. arXiv preprint arXiv:2211.15866, 2022.
- [68] Mittal M, Mohan R, Burgard W, et al. Vision-based autonomous uav navigation and landing for urban search and rescue[M]//Asfour T, Mayer N, Šegvić S, et al. Robotics Research. ISRR 2019, SPAR 2020. Springer Proceedings in Advanced Robotics: volume 20. Springer, Cham, 2022: 575–592.
- [69] Sandino J, Maire F, Caccetta P, et al. Drone-based autonomous motion planning system for outdoor environments under object detection uncertainty[J]. Remote Sensing, 2021, 13(21): 4481.
- [70] Kumar R, Singh L, Tiwari R. Novel reinforcement learning guided enhanced variable weight grey wolf optimization (rlv-gwo) algorithm for multi-uav path planning[J]. Wireless Personal Communications, 2023, 131(3): 2093–2123.
- [71] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100–107.
- [72] Koenig S, Likhachev M. D* lite[C]//Proceedings of the AAAI Conference on Artificial Intelligence: volume 2. 2002: 476–483.